Slip 1:-

Section I: Data Structures-II **[15 Marks]**

Q1) Implement a Binary search tree (BST) library (btree.h) with operations – create, search, insert, inorder, preorder and postorder. Write a menu driven program that performs the above operations.

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following Entities and their Relationships for Project-Employee database.
Project (pno integer, pname char (30), ptype char (20), duration integer)
Employee (eno integer, ename char (20), qualification char (15), joining_date date)

The Relationship between Project and Employee is **many to many** with descriptive attribute start_date date, no_of_hours_worked integer.
Constraints: Primary Key, duration should be greater than zero, pname should not be null.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a stored function to accept eno as input parameter and count number of projects on which that employee is working.

OR

1) Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message

2) Write a procedure to display addition, subtraction and multiplication of three numbers.

[5 marks]

Ds) 2)

```
#include<stdio.h>


int main()
{
   int n,i,j;
   int adj[10][10];


   printf("Enter number of vertices: ");
   scanf("%d",&n);


   /* Initialize matrix with 0 */
   for(i=0;i<n;i++)
   {
```

```
    for(j=0;j<n;j++)
    {
        adj[i][j]=0;
    }
}

/* Accept edges */
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        if(i!=j)
        {
            printf("Is there an edge between %d and %d (1/0): ",i+1,j+1);
            scanf("%d",&adj[i][j]);
        }
    }
}

/* Display adjacency matrix */
printf("\nAdjacency Matrix:\n");

for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        printf("%d ",adj[i][j]);
    }
    printf("\n");
}
```

```
    return 0;
}
```

DBMS:-

```
CREATE TABLE Project (
    pno INTEGER PRIMARY KEY,
    pname CHAR(30) NOT NULL,
    ptype CHAR(20),
    duration INTEGER CHECK (duration > 0)
);

CREATE TABLE Employee (
    eno INTEGER PRIMARY KEY,
    ename CHAR(20),
    qualification CHAR(15),
    joining_date DATE
);

CREATE TABLE Project_Employee (
    pno INTEGER REFERENCES Project(pno),
    eno INTEGER REFERENCES Employee(eno),
    start_date DATE,
    no_of_hours_worked INTEGER,
    PRIMARY KEY (pno, eno)
);
```

1A)

```
CREATE OR REPLACE FUNCTION count_projects(emp_no INT) RETURNS INTEGER LANGUAGE plpgsql
AS $$
DECLARE
    proj_count INTEGER;
```

```
BEGIN

    SELECT COUNT(*) INTO proj_count
    FROM Project_Employee
    WHERE eno = emp_no;

    RETURN proj_count;

END;
$$;
```

2)

```
CREATE OR REPLACE PROCEDURE calc_numbers(a INT, b INT, c INT) LANGUAGE plpgsql AS $$
DECLARE
    sum_val INT;
    sub_val INT;
    mul_val INT;
BEGIN

    sum_val := a + b + c;
    sub_val := a - b - c;
    mul_val := a * b * c;

    RAISE NOTICE 'Addition = %', sum_val;
    RAISE NOTICE 'Subtraction = %', sub_val;
    RAISE NOTICE 'Multiplication = %', mul_val;

END;
$$;
```

Slip 2:-

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree

OR

Q1) Write a C program that accepts the vertices and edges of a graph. Create and display adjacency list also print indegree, outdegree and total degree of all vertex of graph.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following Entities and their Relationships
Person (pno integer, pname varchar (20), birthdate date, income money)
Area (aname varchar (20), area_type varchar (5))
An area can have **one or more persons** living in it, but a person belongs to exactly one area.

Constraints: Primary Key,area_type can be either **'urban' or 'rural'.**

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a stored function to print total number of persons of a particular area. Accept area name as input parameter.

OR

1) Write a stored function to print sum of income of person living in " " area type. (Accept area type as input parameter). Display appropriate message for invalid area type.

2) Write a procedure to display division of two numbers. Use raise to display error messages for division by zero error. **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};


/* Create BST */
```

```
struct node* create(struct node *root)
{
    int n,i,x;
    struct node *nw,*temp;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Enter value: ");
        scanf("%d",&x);

        nw=(struct node*)malloc(sizeof(struct node));
        nw->data=x;
        nw->left=nw->right=NULL;

        if(root==NULL)
            root=nw;
        else
        {
            temp=root;

            while(1)
            {
                if(x < temp->data)
                {
                    if(temp->left==NULL)
                    {
                        temp->left=nw;
                        break;
```

```
            }
            temp=temp->left;
        }
        else
        {
            if(temp->right==NULL)
            {
                temp->right=nw;
                break;
            }
            temp=temp->right;
        }
    }
  }
 }

 return root;
}

/* Find Minimum */
int findMin(struct node *root)
{
 while(root->left!=NULL)
   root=root->left;

 return root->data;
}

/* Find Maximum */
int findMax(struct node *root)
{
```

```
    while(root->right!=NULL)
        root=root->right;

    return root->data;
}

int main()
{
    struct node *root=NULL;

    root=create(root);

    printf("Minimum value = %d\n",findMin(root));
    printf("Maximum value = %d\n",findMax(root));

    return 0;
}
```

DBMS:-

```
CREATE TABLE Area (
    aname VARCHAR(20) PRIMARY KEY,
    area_type VARCHAR(5) CHECK (area_type IN ('urban','rural'))
);

CREATE TABLE Person (
    pno INTEGER PRIMARY KEY,
    pname VARCHAR(20),
    birthdate DATE,
    income MONEY,
    aname VARCHAR(20) REFERENCES Area(aname)
);
```

1A)

```
CREATE OR REPLACE FUNCTION count_persons(area_name VARCHAR)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
   total_persons INTEGER;
BEGIN

   SELECT COUNT(*) INTO total_persons
   FROM Person
   WHERE aname = area_name;

   RETURN total_persons;

END;
$$;
```

2)

```
CREATE OR REPLACE PROCEDURE divide_numbers(a NUMERIC, b NUMERIC) LANGUAGE plpgsql
AS $$
DECLARE
   result NUMERIC;
BEGIN

   IF b = 0 THEN
      RAISE EXCEPTION 'Error: Division by zero is not allowed';
   END IF;

   result := a / b;
```

```
    RAISE NOTICE 'Division Result = %', result;

END;
$$;
```

Slip 3:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an **adjacency matrix**. Implement function to traverse the graph using Breadth First Search (BFS) traversal.

OR

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following Entities and their Relationship.

Student (s_no integer, s_name char (20), address char (25), class char (10))

Teacher (t_no integer,t_name char (10), qualification char (10),experience integer)

Relationship between Student and Teacher is **Many to Many** with descriptive attribute subject and marks_scored.

Constraints: Primary Key,s_name,t_name should not be null,marks_scored> 0

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a cursor which will accept student number from the user and display student name along with teacher name who taught 'RDBMS' subject.

OR

1) Write a trigger before insert the record of student. If student number is less than or equal to zero give message "Invalid Number".
2) Write a procedure to find minimum and maximum from two numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};


/* Create BST */

struct node* create(struct node *root)

{

   int n,i,x;
```

```
struct node *nw,*temp;

printf("Enter number of nodes: ");
scanf("%d",&n);

for(i=0;i<n;i++)
{
    printf("Enter value: ");
    scanf("%d",&x);

    nw=(struct node*)malloc(sizeof(struct node));
    nw->data=x;
    nw->left=nw->right=NULL;

    if(root==NULL)
        root=nw;
    else
    {
        temp=root;

        while(1)
        {
            if(x < temp->data)
            {
                if(temp->left==NULL)
                {
                    temp->left=nw;
                    break;
                }
                temp=temp->left;
            }
```

```
            else
            {
                if(temp->right==NULL)
                {
                    temp->right=nw;
                    break;
                }
                temp=temp->right;
            }
        }
    }
  }

  return root;
}

/* Find Height */
int height(struct node *root)
{
  int lh,rh;

  if(root==NULL)
    return 0;

  lh = height(root->left);
  rh = height(root->right);

  if(lh > rh)
    return lh + 1;
  else
    return rh + 1;
```

```
}

/* Check Balanced Tree */
int isBalanced(struct node *root)
{
   int lh,rh;

   if(root==NULL)
      return 1;

   lh = height(root->left);
   rh = height(root->right);

   if(abs(lh-rh)<=1 && isBalanced(root->left) && isBalanced(root->right))
      return 1;

   return 0;
}

int main()
{
   struct node *root=NULL;

   root=create(root);

   printf("Height of tree = %d\n",height(root));

   if(isBalanced(root))
      printf("Tree is Balanced\n");
   else
      printf("Tree is NOT Balanced\n");
```

```
    return 0;
}
```

DBMS:-

```
CREATE TABLE Student (
    s_no INTEGER PRIMARY KEY,
    s_name CHAR(20) NOT NULL,
    address CHAR(25),
    class CHAR(10)
);

CREATE TABLE Teacher (
    t_no INTEGER PRIMARY KEY,
    t_name CHAR(10) NOT NULL,
    qualification CHAR(10),
    experience INTEGER
);

CREATE TABLE Student_Teacher (
    s_no INTEGER REFERENCES Student(s_no),
    t_no INTEGER REFERENCES Teacher(t_no),
    subject VARCHAR(20),
    marks_scored INTEGER CHECK (marks_scored > 0),
    PRIMARY KEY (s_no, t_no, subject)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_student_number()
RETURNS TRIGGER
AS $$
BEGIN
```

```
    IF NEW.s_no <= 0 THEN
        RAISE EXCEPTION 'Invalid Number';
    END IF;

    RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_student_no
BEFORE INSERT
ON Student
FOR EACH ROW
EXECUTE FUNCTION check_student_number();
```

2)

```
CREATE OR REPLACE PROCEDURE find_min_max(a INT, b INT)
LANGUAGE plpgsql
AS $$
DECLARE
    min_val INT;
    max_val INT;
BEGIN

    IF a < b THEN
        min_val := a;
        max_val := b;
    ELSE
        min_val := b;
        max_val := a;
```

```
    END IF;

    RAISE NOTICE 'Minimum = %', min_val;
    RAISE NOTICE 'Maximum = %', max_val;

END;
$$;
```

Slip 4:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an **adjacency list.** Implement function to traverse the graph using Depth First Search (DFS) traversal.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following Entities and their Relationships .

Movies (m_name varchar (25), release_year integer, budget money)

Actor (a_name char (30), role char (30), charges money, a_address varchar(30))

Producer (producer_id integer, name char (30), p_address varchar (30))

The relationships are as follows:

Each actor has acted in one or more movies. Each producer has produced many movies and each movie can be produced by more than one producers. Each movie has one or more actors acting in it, in different roles.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a cursor to pass actor name as input parameter and return total number of movies in which given actor is acting.

OR

1) Write a trigger before inserting into movie table to check budget. Budget should be minimum 50 lakh. Display appropriate message.

2) Write a procedure to check the number is positive, negative or zero. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


#define MAX 10


struct node

{

   int vertex;

   struct node *next;

};
```

```
struct node *adj[MAX];
int visited[MAX];

void dfs(int v)
{
    struct node *temp;

    visited[v] = 1;
    printf("\tV[%d]", v+1);

    temp = adj[v];

    while(temp != NULL)
    {
        if(visited[temp->vertex] == 0)
            dfs(temp->vertex);

        temp = temp->next;
    }
}

int main()
{
    struct node *nw, *temp;
    int n,i,j,reply;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
```

```
    adj[i]=NULL;
    visited[i]=0;
}

for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        if(i!=j)
        {
            printf("\nIs edge present between %d & %d (1/0): ",i+1,j+1);
            scanf("%d",&reply);

            if(reply==1)
            {
                nw=(struct node*)malloc(sizeof(struct node));
                nw->vertex=j;
                nw->next=NULL;

                if(adj[i]==NULL)
                    adj[i]=nw;
                else
                {
                    temp=adj[i];
                    while(temp->next!=NULL)
                        temp=temp->next;

                    temp->next=nw;
                }
            }
        }
```

```
        }
    }

    printf("\nDFS Traversal:\n");
    dfs(0);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Movies (
    m_name VARCHAR(25) PRIMARY KEY,
    release_year INTEGER,
    budget MONEY
);

CREATE TABLE Actor (
    a_name CHAR(30) PRIMARY KEY,
    role CHAR(30),
    charges MONEY,
    a_address VARCHAR(30)
);

CREATE TABLE Producer (
    producer_id INTEGER PRIMARY KEY,
    name CHAR(30),
    p_address VARCHAR(30)
);

CREATE TABLE Movie_Actor (
    m_name VARCHAR(25) REFERENCES Movies(m_name),
    a_name CHAR(30) REFERENCES Actor(a_name),
```

```
    PRIMARY KEY(m_name, a_name)
);

CREATE TABLE Movie_Producer (
    m_name VARCHAR(25) REFERENCES Movies(m_name),
    producer_id INTEGER REFERENCES Producer(producer_id),
    PRIMARY KEY(m_name, producer_id)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_movie_budget()
RETURNS TRIGGER
AS $$
BEGIN

    IF NEW.budget < 5000000 THEN
        RAISE EXCEPTION 'Budget must be at least 50 lakh';
    END IF;

    RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_budget
BEFORE INSERT
ON Movies
FOR EACH ROW
EXECUTE FUNCTION check_movie_budget();
```

2)

```
CREATE OR REPLACE PROCEDURE check_number(n INT)
LANGUAGE plpgsql
AS $$
BEGIN

   IF n > 0 THEN
      RAISE NOTICE 'Number is Positive';

   ELSIF n < 0 THEN
      RAISE NOTICE 'Number is Negative';

   ELSE
      RAISE NOTICE 'Number is Zero';
   END IF;

END;
$$;
```

Slip 5:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program which uses Binary search tree library and display the following

i. Node value at each level
ii. Count of node at each level
iii. Total levels in the tree.

OR

Q1) Write a C program for the Implementation of Prim's Minimum spanning tree algorithm.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following Entities and their Relationships for Student-Competition
Student (sreg_no int ,s_name varchar(20), s_class char(10))

Competition (c_no int ,c_name varchar(20), c_type char(10))

Relationship between Student and Competition is many to many with descriptive attributes rank and year.

Constraints: Primary Key, c_type should not be null, c_type can be 'sport' or 'academic'.

Q2) Using above Database Solve the following **[10 Marks]**

1. write a function using cursor to accept year and class to display student name, competition name and rank.

OR

1. Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

2. Write a procedure to find maximum and minimum from three numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#include<stdlib.h>


#define MAX 20


struct node

{

   int data;

   struct node *left,*right;
```

```
};

typedef struct
{
   struct node *data[MAX];
   int front,rear;
}queue;

/* Queue Functions */

void initq(queue *q)
{
   q->front = q->rear = -1;
}

void addq(queue *q, struct node *n)
{
   q->data[++q->rear] = n;
}

struct node* delq(queue *q)
{
   return q->data[++q->front];
}

int isempty(queue *q)
{
   return (q->front == q->rear);
}

/* Create BST */
```

```
struct node* create(struct node *root)
{
    struct node *nw,*temp;
    int n,i,x;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        scanf("%d",&x);

        nw=(struct node*)malloc(sizeof(struct node));
        nw->data=x;
        nw->left=nw->right=NULL;

        if(root==NULL)
            root=nw;
        else
        {
            temp=root;

            while(1)
            {
                if(x < temp->data)
                {
                    if(temp->left==NULL)
                    {
                        temp->left=nw;
                        break;
```

```
            }
            temp=temp->left;
        }
        else
        {
            if(temp->right==NULL)
            {
                temp->right=nw;
                break;
            }
            temp=temp->right;
        }
    }
  }
}

  return root;
}

/* Level Order Traversal */

void levelorder(struct node *root)
{
  queue q;
  struct node *temp,*marker=NULL;
  int level=0,count=0,total_levels=0;

  initq(&q);

  addq(&q,root);
  addq(&q,marker);
```

```
printf("\nLevel %d : ",level);

while(!isempty(&q))
{
   temp=delq(&q);

   if(temp==marker)
   {
      printf("\nCount of nodes at level %d = %d\n",level,count);

      count=0;
      level++;
      total_levels++;

      if(!isempty(&q))
      {
         addq(&q,marker);
         printf("\nLevel %d : ",level);
      }
   }
   else
   {
      printf("%d ",temp->data);
      count++;

      if(temp->left)
         addq(&q,temp->left);

      if(temp->right)
         addq(&q,temp->right);
```

```
        }
    }

    printf("\nTotal Levels in Tree = %d\n",total_levels);
}

int main()
{
    struct node *root=NULL;

    root=create(root);

    levelorder(root);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Student (
    sreg_no INT PRIMARY KEY,
    s_name VARCHAR(20),
    s_class CHAR(10)
);

CREATE TABLE Competition (
    c_no INT PRIMARY KEY,
    c_name VARCHAR(20),
    c_type CHAR(10) NOT NULL CHECK (c_type IN ('sport','academic'))
);

CREATE TABLE Student_Competition (
```

```
    sreg_no INT REFERENCES Student(sreg_no),
    c_no INT REFERENCES Competition(c_no),
    rank INT,
    year INT,
    PRIMARY KEY (sreg_no, c_no, year)
);
```

1B)

```
CREATE OR REPLACE FUNCTION competition_update_msg()
RETURNS TRIGGER
AS $$
BEGIN
    RAISE NOTICE 'Competition record is being updated';
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_comp_update
BEFORE UPDATE
ON Competition
FOR EACH ROW
EXECUTE FUNCTION competition_update_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE find_min_max_three(a INT, b INT, c INT)
LANGUAGE plpgsql
AS $$
DECLARE
    max_val INT;
    min_val INT;
```

```
BEGIN

    max_val := GREATEST(a,b,c);
    min_val := LEAST(a,b,c);

    RAISE NOTICE 'Maximum = %', max_val;
    RAISE NOTICE 'Minimum = %', min_val;

END;
$$;
```

Slip 6:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

OR

Q1) Write a C program which uses Binary search tree library and implements following function with recursion:

int copy(T) – create another BST which is exact copy of BST which is passed as parameter.

int compare(T1, T2) – compares two binary search trees and returns 1 if they are equal and 0 otherwise.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships

BUS (bus_no int , capacity int , depot_name varchar(20))

ROUTE (route_no int, source char(20), destination char(20),no_of_stations int)

DRIVER (driver_no int , driver_name char(20), license_no int, address char(20), d_ageint , salary float)

The relationships are as: BUS_ROUTE: M-1

BUS_DRIVER: M-M with descriptive attributes Date of duty allotted and Shift itcan be 1 (Morning) or 2 (Evening).

Constraints: 1. License_no is unique.

2. Bus capacity is not null

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a stored function to accept the bus number and print driver name allotted to that bus.

OR

1. Write a stored function to accept depot name and display driver details having age more than 50.

2. Write a procedure to find number is even or odd. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};
```

```
/* Create BST */
struct node* create(struct node *T)
{
    struct node *nw,*temp;
    int n,i,x;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Enter value: ");
        scanf("%d",&x);

        nw=(struct node*)malloc(sizeof(struct node));
        nw->data=x;
        nw->left=nw->right=NULL;

        if(T==NULL)
            T=nw;
        else
        {
            temp=T;

            while(1)
            {
                if(x < temp->data)
                {
                    if(temp->left==NULL)
                    {
                        temp->left=nw;
```

```
                break;
            }
            temp=temp->left;
        }
        else
        {
            if(temp->right==NULL)
            {
                temp->right=nw;
                break;
            }
            temp=temp->right;
        }
    }
  }
 }

 return T;
}

/* Copy Tree */
struct node* copy(struct node *T)
{
 struct node *nw;

 if(T==NULL)
   return NULL;

 nw=(struct node*)malloc(sizeof(struct node));
 nw->data=T->data;
```

```
    nw->left=copy(T->left);
    nw->right=copy(T->right);

    return nw;
}

/* Compare Trees */
int compare(struct node *T1, struct node *T2)
{
    if(T1==NULL && T2==NULL)
        return 1;

    if(T1!=NULL && T2!=NULL)
    {
        if(T1->data==T2->data &&
          compare(T1->left,T2->left) &&
          compare(T1->right,T2->right))
           return 1;
    }

    return 0;
}

/* Inorder Traversal */
void inorder(struct node *T)
{
    if(T!=NULL)
    {
        inorder(T->left);
        printf("%d ",T->data);
        inorder(T->right);
```

```
    }
}

int main()
{
    struct node *T=NULL,*T2=NULL;

    T=create(T);        // create original tree
    T2=copy(T);         // copy tree

    printf("\nOriginal Tree (Inorder): ");
    inorder(T);

    printf("\nCopied Tree (Inorder): ");
    inorder(T2);

    if(compare(T,T2))
        printf("\nTrees are Equal\n");
    else
        printf("\nTrees are NOT Equal\n");

    return 0;
}
```

DBMS:-

```
CREATE TABLE BUS (
    bus_no INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(20)
);
```

```
CREATE TABLE ROUTE (
   route_no INT PRIMARY KEY,
   source CHAR(20),
   destination CHAR(20),
   no_of_stations INT
);

CREATE TABLE DRIVER (
   driver_no INT PRIMARY KEY,
   driver_name CHAR(20),
   license_no INT UNIQUE,
   address CHAR(20),
   d_age INT,
   salary FLOAT
);
-- BUS_ROUTE (Many buses belong to one route)
CREATE TABLE BUS_ROUTE (
   bus_no INT REFERENCES BUS(bus_no),
   route_no INT REFERENCES ROUTE(route_no),
   PRIMARY KEY(bus_no)
);

-- BUS_DRIVER (Many-to-Many)
CREATE TABLE BUS_DRIVER (
   bus_no INT REFERENCES BUS(bus_no),
   driver_no INT REFERENCES DRIVER(driver_no),
   duty_date DATE,
   shift INT CHECK (shift IN (1,2)),
   PRIMARY KEY(bus_no, driver_no)
);
```

1A)

```
CREATE OR REPLACE FUNCTION get_driver_by_bus(b_no INT)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
   rec RECORD;
BEGIN

   FOR rec IN
      SELECT d.driver_name
      FROM DRIVER d
      JOIN BUS_DRIVER bd ON d.driver_no = bd.driver_no
      WHERE bd.bus_no = b_no
   LOOP
      RAISE NOTICE 'Driver Name: %', rec.driver_name;
   END LOOP;

END;
$$;
```

2)

```
CREATE OR REPLACE PROCEDURE check_even_odd(n INT)
LANGUAGE plpgsql
AS $$
BEGIN

   IF n % 2 = 0 THEN
      RAISE NOTICE 'Number is Even';
```

```
    ELSE
        RAISE NOTICE 'Number is Odd';
    END IF;

END;
$$;
```

Slip 7:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the implementation of Topological sorting.

OR

Q1) Write a C program for the implementation of Dijkstra's shortest path algorithm for finding shortest path from a given source vertex using adjacency cost matrix.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships
Branch (br_id integer, br_name char (30), br_city char (10))
Customer (cno integer, c_name char (20), caddr char (35), city char (20))
Loan_application(lno integer, l_amt_required money, l_amt_approved money, l_date date)
Relationship between Branch, Customer and Loan_application is Ternary. Ternary (br_id integer, cno integer, lno integer) Constraints: Primary Key, l_amt_required should be greater than zero.

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a stored function using cursor to count the number of customers of particular branch. (Accept branch name as input parameter).

OR

1. Write a trigger before insert record of customer. If the customer number is less than or equal to zero, then give the appropriate message.

2. Write a procedure to accept value of n and display sum of first n numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#define MAX 10


typedef struct {

   int data[MAX];

   int top;

}STACK;


void initStack(STACK *ps) {

   ps->top = -1;

}
```

```
void push(STACK *ps, int num)
{
    ps->top++;
    ps->data[ps->top] = num;
}

int pop(STACK *ps)
{
    return (ps->data[ps->top--]);
}

int isempty(STACK *ps)
{
    return (ps->top == -1);
}

int isfull(STACK *ps)
{
    return (ps->top == MAX - 1);
}

void topologicalSort(int m[10][10], int n)
{
    int i,j,v,w;
    int visited[10]={0};
    int indegree[10]={0};
    STACK s;
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
```

```
            indegree[i] += m[j][i];
        }
    }
    initStack(&s);

    while(1)
    {
        for(v=0;v<n;v++)
        {
            if((visited[v]==0) && (indegree[v]==0))
            {
                visited[v]=1;
                push(&s,v);
                printf("v[%d]\t",v+1);
            }
        }
        if(isempty(&s))
            break;
        v=pop(&s);
        for(w=0;w<n;w++)
        {
            if(m[v][w]==1)
                indegree[w]= indegree[w]-1;
        }
    }
}

int main()
{
    int m[10][10],n,i,j;
    printf("Enter the number of vertices:");
```

```
    scanf("%d",&n);
    printf("Enter the adjacency matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&m[i][j]);
        }
    }
    printf("Topological order of the vertices:\n");
    topologicalSort(m,n);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Branch (
br_id INTEGER PRIMARY KEY,
br_name CHAR(30),
br_city CHAR(10)
);

CREATE TABLE Customer (
cno INTEGER PRIMARY KEY,
c_name CHAR(20),
caddr CHAR(35),
city CHAR(20)
);

CREATE TABLE Loan_application (
    lno INTEGER PRIMARY KEY,
    l_amt_required MONEY CHECK (l_amt_required > 0::money),
    l_amt_approved MONEY,
    l_date DATE
);
```

```
CREATE TABLE Ternary (
   br_id INTEGER REFERENCES Branch(br_id),
   cno INTEGER REFERENCES Customer(cno),
   lno INTEGER REFERENCES Loan_application(lno),
   PRIMARY KEY (br_id, cno, lno)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_customer_number()
RETURNS TRIGGER
AS $$
BEGIN

   IF NEW.cno <= 0 THEN
      RAISE EXCEPTION 'Customer number must be greater than zero';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_customer
BEFORE INSERT
ON Customer
FOR EACH ROW
EXECUTE FUNCTION check_customer_number();
```

2)

```
CREATE OR REPLACE PROCEDURE sum_n(n INT)
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 1;
   s INT := 0;
BEGIN

   WHILE i <= n LOOP
      s := s + i;
      i := i + 1;
   END LOOP;

   RAISE NOTICE 'Sum of first % numbers = %', n, s;

END;
$$;
```

Slip 8:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a program to sort n randomly generated elements using Heapsort method.

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships
TRAIN: (train_no int, train_name varchar(20), depart_time time , arrival_time time, source_stn varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int)
PASSENGER : (passenger_id int, passenger_name varchar(20), address varchar(30), age int ,gender char)
Relationships:

Train _Passenger: M-M relationship named ticket with descriptive attributes as follows

TICKET: (train_no int, passenger_id int, ticket_no int ,bogie_no int, no_of_berths int ,tdate date , ticket_amt decimal(7,2),status char)

Constraints: The status of a berth can be 'W' (waiting) or 'C' (confirmed).

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a stored function to display the ticket details of a train. (Accept train name as input parameter). Raise an exception in case of invalid train name.

OR

1.Write a trigger after insert on passenger to display message "Age above 5 will be charged full fare" if age of passenger is more than 5.

2. Write a procedure that accept two numbers and display addition, subtraction and multiplication of two numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>


int main()

{

   int n, e, i, j;

   int adj[10][10];

   int v1, v2;


   printf("Enter number of vertices: ");
```

```
scanf("%d",&n);

printf("Enter number of edges: ");
scanf("%d",&e);

/* Initialize matrix with 0 */
for(i=0;i<n;i++)
{
   for(j=0;j<n;j++)
   {
      adj[i][j]=0;
   }
}

/* Accept edges */
printf("Enter edges (vertex1 vertex2):\n");
for(i=0;i<e;i++)
{
   scanf("%d%d",&v1,&v2);

   adj[v1][v2]=1;
   adj[v2][v1]=1;   // for undirected graph
}

/* Display adjacency matrix */
printf("\nAdjacency Matrix:\n");

for(i=0;i<n;i++)
{
   for(j=0;j<n;j++)
   {
```

```
            printf("%d ",adj[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

DBMS:-

```
CREATE TABLE TRAIN (
    train_no INT PRIMARY KEY,
    train_name VARCHAR(20),
    depart_time TIME,
    arrival_time TIME,
    source_stn VARCHAR(20),
    dest_stn VARCHAR(20),
    no_of_res_bogies INT,
    bogie_capacity INT
);

CREATE TABLE PASSENGER (
    passenger_id INT PRIMARY KEY,
    passenger_name VARCHAR(20),
    address VARCHAR(30),
    age INT,
    gender CHAR(1)
);
CREATE TABLE TICKET (
    train_no INT REFERENCES TRAIN(train_no),
    passenger_id INT REFERENCES PASSENGER(passenger_id),
    ticket_no INT,
```

```
    bogie_no INT,
    no_of_berths INT,
    tdate DATE,
    ticket_amt DECIMAL(7,2),
    status CHAR(1) CHECK (status IN ('W','C')),
    PRIMARY KEY(train_no, passenger_id, ticket_no)
);
```

1B)

```
CREATE OR REPLACE FUNCTION passenger_age_check()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.age > 5 THEN
        RAISE NOTICE 'Age above 5 will be charged full fare';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_passenger_age
AFTER INSERT
ON PASSENGER
FOR EACH ROW
EXECUTE FUNCTION passenger_age_check();
```

2)

```
CREATE OR REPLACE PROCEDURE operations(a INT, b INT)
LANGUAGE plpgsql
AS $$
DECLARE
   add_res INT;
   sub_res INT;
   mul_res INT;
BEGIN

   add_res := a + b;
   sub_res := a - b;
   mul_res := a * b;

   RAISE NOTICE 'Addition = %', add_res;
   RAISE NOTICE 'Subtraction = %', sub_res;
   RAISE NOTICE 'Multiplication = %', mul_res;

END;
$$;
```

Slip 9:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to implement a hash table using **separate chaining** with linked lists with following operation.

a. Insert a key
b. Search a key
c. Display the hash table

OR

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following database maintained by a school about students and competitions.
STUDENT (sreg_no int , name char(30), class char(10))
COMPETITION (c_no int , name char(20), C_type char(15))

The relationship is as follows:

STUDENT-COMPETITION: **M-M** with described attributes rank, year and prize(int).

Q2) Using above Database Solve the following **[10 Marks]**

1. Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

OR

1. Write a function using cursor which accepts a name of student and returns the total no of prizes won by that student in the year 2020.

2. Write a procedure to display division of two numbers use raise to display divided by zero exception messages. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#define ts 10


typedef struct node

{

   int key,chain;

}HT;


/* Hash Function */

int hf(int key,int i)
```

```
{
    return ((key%10+1)%10);
}

/* Initialize Hash Table */
void init(HT ht[ts])
{
    int i;
    for(i=0;i<ts;i++)
    {
        ht[i].key=-1;
        ht[i].chain=-1;
    }
}

/* Insert Key */
void insert(HT ht[ts],int key)
{
    int index = hf(key,0);

    if(ht[index].key==-1)
    {
        ht[index].key=key;
        return;
    }

    int i;
    for(i=0;i<ts;i++)
    {
        if(ht[i].key==-1)
        {
```

```
            ht[i].key=key;

            int temp=index;
            while(ht[temp].chain!=-1)
                temp=ht[temp].chain;

            ht[temp].chain=i;
            return;
        }
    }

    printf("Hash table full\n");
}

/* Search Key */
int search(HT ht[ts],int key)
{
    int index=hf(key,0);

    while(index!=-1)
    {
        if(ht[index].key==key)
            return index;

        index=ht[index].chain;
    }

    return -1;
}

/* Display Hash Table */
```

```
void showtable(HT ht[ts])
{
    int i;
    for(i=0;i<ts;i++)
        printf("%d.[%d][%d]\n",i,ht[i].key,ht[i].chain);
}

/* Main Function */
int main()
{
    HT ht[ts];
    int key,ch;

    init(ht);

    printf("1.Insert\n2.Search\n3.Display\n4.Exit\n");

    while(1)
    {
        printf("Enter choice: ");
        scanf("%d",&ch);

        switch(ch)
        {
            case 1:
                printf("Enter key: ");
                scanf("%d",&key);
                insert(ht,key);
                break;

            case 2:
```

```
            printf("Enter key: ");
            scanf("%d",&key);

            int pos=search(ht,key);

            if(pos==-1)
                printf("Key not found\n");
            else
                printf("Key found at index %d\n",pos);

            break;

        case 3:
            showtable(ht);
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
    }
  }
}
```

DBMS:-

```
CREATE TABLE STUDENT (
  sreg_no INT PRIMARY KEY,
  name CHAR(30),
  class CHAR(10)
);
```

```
CREATE TABLE COMPETITION (
   c_no INT PRIMARY KEY,
   name CHAR(20),
   c_type CHAR(15)
);

CREATE TABLE STUDENT_COMPETITION (
   sreg_no INT REFERENCES STUDENT(sreg_no),
   c_no INT REFERENCES COMPETITION(c_no),
   rank INT,
   year INT,
   prize INT,
   PRIMARY KEY(sreg_no,c_no,year)
);
```

1A)

```
CREATE OR REPLACE FUNCTION comp_update_msg()
RETURNS TRIGGER
AS $$
BEGIN
   RAISE NOTICE 'Competition record is being updated';
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_comp_update
BEFORE UPDATE
ON COMPETITION
FOR EACH ROW
EXECUTE FUNCTION comp_update_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE divide_numbers(a INT, b INT)
LANGUAGE plpgsql
AS $$
DECLARE
   result FLOAT;
BEGIN

   IF b = 0 THEN
      RAISE EXCEPTION 'Division by zero is not allowed';
   END IF;

   result := a / b;

   RAISE NOTICE 'Result = %', result;

END;
$$;
```

Slip 10:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the implementation of Floyd Warshall's algorithm for finding all pairs shortest path using adjacency cost matrix.

OR

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships
Book(b_no int, b_name varchar (20), pub_name varchar (10), b_price float)
Author (a_no int, a_name varchar (20), qualification varchar (15), address varchar (15))

Relationship between Book and Author is many to many.
Constraints: Primary Key,pub_name should not be null.

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a stored function to display the book details written by author . (Accept Author name as input parameter). Raise an exception in case of invalid author name.

OR

1. Write a trigger after insert on book to display message "prize is so high" if book price is more than 1000.
2. Write a procedure to display all even numbers from 1 to 50. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};


/* Create BST */

struct node* create(struct node *T)

{
```

```
int n,i,x;
struct node *nw,*temp;

printf("Enter number of nodes: ");
scanf("%d",&n);

for(i=0;i<n;i++)
{
   printf("Enter value: ");
   scanf("%d",&x);

   nw=(struct node*)malloc(sizeof(struct node));
   nw->data=x;
   nw->left=nw->right=NULL;

   if(T==NULL)
      T=nw;
   else
   {
      temp=T;

      while(1)
      {
         if(x < temp->data)
         {
            if(temp->left==NULL)
            {
               temp->left=nw;
               break;
            }
            temp=temp->left;
```

```
            }
            else
            {
                if(temp->right==NULL)
                {
                    temp->right=nw;
                    break;
                }
                temp=temp->right;
            }
        }
    }
  }

  return T;
}

/* Find Minimum */
int findMin(struct node *T)
{
  while(T->left != NULL)
    T = T->left;

  return T->data;
}

/* Find Maximum */
int findMax(struct node *T)
{
  while(T->right != NULL)
    T = T->right;
```

```
    return T->data;
}

int main()
{
    struct node *T=NULL;

    T=create(T);

    printf("Minimum value = %d\n",findMin(T));
    printf("Maximum value = %d\n",findMax(T));

    return 0;
}
```

DBMS:-

```
CREATE TABLE Book (
    b_no INT PRIMARY KEY,
    b_name VARCHAR(20),
    pub_name VARCHAR(10) NOT NULL,
    b_price FLOAT
);

CREATE TABLE Author (
    a_no INT PRIMARY KEY,
    a_name VARCHAR(20),
    qualification VARCHAR(15),
    address VARCHAR(15)
);
```

```
CREATE TABLE Book_Author (
   b_no INT REFERENCES Book(b_no),
   a_no INT REFERENCES Author(a_no),
   PRIMARY KEY (b_no,a_no)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_book_price()
RETURNS TRIGGER
AS $$
BEGIN

   IF NEW.b_price > 1000 THEN
      RAISE NOTICE 'Price is so high';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_book_price
AFTER INSERT
ON Book
FOR EACH ROW
EXECUTE FUNCTION check_book_price();
```

2)

```
CREATE OR REPLACE PROCEDURE show_even_numbers()
LANGUAGE plpgsql
AS $$
```

```
DECLARE
   i INT := 2;
BEGIN

   WHILE i <= 50 LOOP
      RAISE NOTICE '%', i;
      i := i + 2;
   END LOOP;

END;
$$;
```

Slip 11:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a program which uses binary search tree library and counts the total nodes and total leaf nodes in the tree.

int count(T) – returns the total number of nodes from BST
int countLeaf(T) – returns the total number of leaf nodes from BST

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an **adjacency list.** Implement function to traverse the graph using Breadth First Search (BFS) traversal.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships

Student (rollno, s_name , class)

Subject (scode , subject_name)

Relationship between Student and Subject is **many to many** with descriptive attribute marks.

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a stored function using cursor to calculate total marks of each student and display it.

OR

1. Define a trigger before deleting a student record from student table. Raise a notice and display message "student record is being deleted".

2. Write a procedure to find sum and average of first n numbers using conditional loop(while) **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};


/* Create BST */
```

```
struct node* create(struct node *T)
{
    struct node *nw,*temp;
    int n,i,x;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Enter value: ");
        scanf("%d",&x);

        nw=(struct node*)malloc(sizeof(struct node));
        nw->data=x;
        nw->left=nw->right=NULL;

        if(T==NULL)
            T=nw;
        else
        {
            temp=T;

            while(1)
            {
                if(x < temp->data)
                {
                    if(temp->left==NULL)
                    {
                        temp->left=nw;
                        break;
```

```
            }
            temp=temp->left;
        }
        else
        {
            if(temp->right==NULL)
            {
                temp->right=nw;
                break;
            }
            temp=temp->right;
        }
    }
  }
 }

 return T;
}

/* YOUR FUNCTION (UNCHANGED) */
int countNode(struct node *T)
{
static int cnt=0;
struct node *temp=T;

if(temp!=NULL)
{
cnt++;
countNode(temp->left);
countNode(temp->right);
}
```

```
return cnt;
}

/* YOUR FUNCTION (UNCHANGED) */
int leafNode(struct node *T)
{
static int lcnt=0;
struct node *temp=T;
if(temp == NULL)
return 0;

if(temp!=NULL)
{
if((temp->left==NULL)&&(temp->right==NULL))
lcnt++;
leafNode(temp->left);
leafNode(temp->right);
}
return lcnt;
}

int main()
{
struct node *T=NULL;

T=create(T);

printf("Total nodes = %d\n",countNode(T));
printf("Total leaf nodes = %d\n",leafNode(T));

return 0;
```

```
}
```

DBMS:-

```
CREATE TABLE Student (
   rollno INT PRIMARY KEY,
   s_name VARCHAR(30),
   class VARCHAR(10)
);
CREATE TABLE Subject (
   scode INT PRIMARY KEY,
   subject_name VARCHAR(30)
);
CREATE TABLE Student_Subject (
   rollno INT REFERENCES Student(rollno),
   scode INT REFERENCES Subject(scode),
   marks INT,
   PRIMARY KEY(rollno, scode)
);
```

1B)

```
CREATE OR REPLACE FUNCTION student_delete_msg()
RETURNS TRIGGER
AS $$
BEGIN
   RAISE NOTICE 'Student record is being deleted';
   RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_student_delete
BEFORE DELETE
```

```
ON Student
FOR EACH ROW
EXECUTE FUNCTION student_delete_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE sum_avg_n(n INT)
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 1;
   sum_val INT := 0;
   avg_val FLOAT;
BEGIN

   WHILE i <= n LOOP
      sum_val := sum_val + i;
      i := i + 1;
   END LOOP;

   avg_val := sum_val / n;

   RAISE NOTICE 'Sum = %', sum_val;
   RAISE NOTICE 'Average = %', avg_val;

END;
$$;
```

Slip 12:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a program to implement various types of hash functions which are used to place the data in a hash table

a. Division Method
b. Mid Square Method
c. Digit Folding Method

Accept n values from the user and display appropriate message in case of collision for each of the above functions.

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an adjacency list. Implement function to traverse the graph using Depth First Search (DFS) traversal.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Project-Employee database maintained by a company which stores the details of the projects assigned to the employees.
Following are the tables:
PROJECT (pno integer, p_name char (30), ptype char(20),duration integer)
EMPLOYEE (eno integer, e_name char (20), qualification char (15), joindate date)

The relationships are as follows:

PROJECT-EMPLOYEE:**M-M** Relationship, with descriptive attributes as start_date (date), no_of_hours_worked (integer).

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message.

OR

1. Write a stored function to accept project name as input and print the names of employees working on the project. Raise an exception for an invalid project name.

2. Write a procedure to count odd numbers from given range (m to n) (for). **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int vertex;

   struct node *next;

};
```

```
int visited[10];

/* read graph */
int read_graph(struct node *adj_list[10], int n)
{
    int i,j,reply;
    struct node *temp,*nw;

    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            if(i==j)
                continue;

            printf("Is there edge between %d and %d: ",i,j);
            scanf("%d",&reply);

            if(reply==1)
            {
                nw=(struct node*)malloc(sizeof(struct node));
                nw->vertex=j;
                nw->next=NULL;

                if(adj_list[i]==NULL)
                    adj_list[i]=nw;
                else
                {
                    temp=adj_list[i];
                    while(temp->next!=NULL)
                        temp=temp->next;
```

```
                temp->next=nw;
            }
        }
    }
  }

  return 0;
}

/* DFS */
void dfs(struct node *adj_list[10], int v)
{
  struct node *temp;

  visited[v]=1;
  printf("%d ",v);

  temp=adj_list[v];

  while(temp!=NULL)
  {
    if(visited[temp->vertex]==0)
      dfs(adj_list,temp->vertex);

    temp=temp->next;
  }
}

int main()
{
```

```
    struct node *adj_list[10];
    int n,i,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    for(i=1;i<=n;i++)
    {
        adj_list[i]=NULL;
        visited[i]=0;
    }

    read_graph(adj_list,n);

    printf("Enter starting vertex: ");
    scanf("%d",&start);

    printf("DFS Traversal:\n");
    dfs(adj_list,start);

    return 0;
}
```

DBMS:-

```
CREATE TABLE PROJECT (
    pno INTEGER PRIMARY KEY,
    p_name CHAR(30),
    ptype CHAR(20),
    duration INTEGER
);
```

```
CREATE TABLE EMPLOYEE (
   eno INTEGER PRIMARY KEY,
   e_name CHAR(20),
   qualification CHAR(15),
   joindate DATE
);
```


```
CREATE TABLE EMPLOYEE (
   eno INTEGER PRIMARY KEY,
   e_name CHAR(20),
   qualification CHAR(15),
   joindate DATE
);
```


1A)

```
CREATE OR REPLACE FUNCTION check_project_duration()
RETURNS TRIGGER
AS $$
BEGIN

   IF NEW.duration <= 0 THEN
      RAISE EXCEPTION 'Duration must be greater than zero';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_duration
BEFORE INSERT
```

```
ON PROJECT
FOR EACH ROW
EXECUTE FUNCTION check_project_duration();
```

2)

```
CREATE OR REPLACE PROCEDURE count_odd(m INT, n INT)
LANGUAGE plpgsql
AS $$
DECLARE
   i INT;
   cnt INT := 0;
BEGIN

   FOR i IN m..n LOOP
      IF i % 2 <> 0 THEN
         cnt := cnt + 1;
      END IF;
   END LOOP;

   RAISE NOTICE 'Total odd numbers between % and % = %', m, n, cnt;

END;
$$;
```

Slip 13:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to implement a hash table using **separate chaining** with linked lists with following operation.

a. Insert a key
b. Search a key
c. Display the hash table

OR

Q1) Write a C program that accepts the vertices and edges of a graph. Create and display adjacency list also print indegree, outdegree and total degree of all vertex of graph.

Section II: Database Management Systems II **[15 Marks]**

Consider the following database of Bus transport system. Many buses run on one route. Drivers are allotted to the buses shift-wise.

Following are the tables:

BUS (bus_no int , capacity int , depot_name varchar(20))
ROUTE (route_no int, source char(20), destination char(20),no_of_stations int)
DRIVER (driver_no int , driver_name char(20), license_no int, address char(20), d_age int , salary float)

The relationships are as follows:

BUS_ROUTE : **M-1**
BUS_DRIVER : **M-M** with descriptive attributes Date of duty allotted and Shift – it can be 1 (Morning) 0r 2 (Evening).

Constraints: 1. License no is unique.
2. Bus capacity is not null.

Q2) Using above Database Solve the following **[10 Marks]**

1. Define a trigger before insert the record of driver if the age is not between 18 and 50, raise an error message "invalid entry".

OR

1. Write a stored function to accept the bus_no and date and print its allotted drivers. Raise an exception in case of invalid bus number.

2. Write a procedure to search the given number is in given range. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


#define MAX 10


struct node

{

   int vertex;
```

```
    struct node *next;
};

void create_graph(struct node *adj[MAX], int n)
{
    int i,j,reply;
    struct node *nw,*temp;

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(i!=j)
            {
                printf("\nIs edge present between %d & %d (1/0): ",i+1,j+1);
                scanf("%d",&reply);

                if(reply==1)
                {
                    nw=(struct node*)malloc(sizeof(struct node));
                    nw->vertex=j;
                    nw->next=NULL;

                    if(adj[i]==NULL)
                        adj[i]=nw;
                    else
                    {
                        temp=adj[i];
                        while(temp->next!=NULL)
                            temp=temp->next;
```

```
                    temp->next=nw;
                }
            }
        }
    }
  }
}

void display_graph(struct node *adj[MAX], int n)
{
  int i;
  struct node *temp;

  printf("\nAdjacency List:\n");

  for(i=0;i<n;i++)
  {
    printf("V[%d] -> ",i+1);

    temp=adj[i];

    while(temp!=NULL)
    {
      printf("V[%d] ",temp->vertex+1);
      temp=temp->next;
    }

    printf("\n");
  }
}
```

```
void indegree_outdegree(struct node *adj[MAX], int n)
{
   int indeg[MAX]={0}, outdeg[MAX]={0};
   int i;
   struct node *temp;

   for(i=0;i<n;i++)
   {
      temp=adj[i];

      while(temp!=NULL)
      {
         outdeg[i]++;
         indeg[temp->vertex]++;
         temp=temp->next;
      }
   }

   printf("\nVertex\tIndegree\tOutdegree\tTotal Degree\n");

   for(i=0;i<n;i++)
   {
      printf("V[%d]\t%d\t\t%d\t\t%d\n",
      i+1, indeg[i], outdeg[i], indeg[i]+outdeg[i]);
   }
}

int main()
{
   struct node *adj[MAX];
   int n,i;
```

```
    printf("Enter number of vertices: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        adj[i]=NULL;

    create_graph(adj,n);
    display_graph(adj,n);
    indegree_outdegree(adj,n);

    return 0;
}
```

DBMS:-

```
CREATE TABLE BUS (
    bus_no INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(20)
);

CREATE TABLE ROUTE (
    route_no INT PRIMARY KEY,
    source CHAR(20),
    destination CHAR(20),
    no_of_stations INT
);

CREATE TABLE DRIVER (
    driver_no INT PRIMARY KEY,
    driver_name CHAR(20),
```

```
    license_no INT UNIQUE,
    address CHAR(20),
    d_age INT,
    salary FLOAT
);

CREATE TABLE BUS_ROUTE (
    bus_no INT REFERENCES BUS(bus_no),
    route_no INT REFERENCES ROUTE(route_no),
    PRIMARY KEY(bus_no)
);

CREATE TABLE BUS_DRIVER (
    bus_no INT REFERENCES BUS(bus_no),
    driver_no INT REFERENCES DRIVER(driver_no),
    duty_date DATE,
    shift INT CHECK (shift IN (1,2)),
    PRIMARY KEY(bus_no, driver_no, duty_date)
);
```

1A)

```
CREATE OR REPLACE FUNCTION check_driver_age()
RETURNS TRIGGER
AS $$
BEGIN

    IF NEW.d_age < 18 OR NEW.d_age > 50 THEN
        RAISE EXCEPTION 'Invalid entry';
    END IF;
```

```
    RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_driver_age
BEFORE INSERT
ON DRIVER
FOR EACH ROW
EXECUTE FUNCTION check_driver_age();
```

2)

```
CREATE OR REPLACE PROCEDURE check_range(num INT, m INT, n INT)
LANGUAGE plpgsql
AS $$
BEGIN

    IF num BETWEEN m AND n THEN
        RAISE NOTICE 'Number % is within the range', num;
    ELSE
        RAISE NOTICE 'Number % is outside the range', num;
    END IF;

END;
$$;
```

Slip 14:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to maintain a phonebook using Binary Search Tree by name where each node contains contact name and phone number.

OR

Q1) Write a C program to implement a hash table using **open addressing with linear probing**. perform the following operations. (Assume all keys are positive integers)

a. Insert
b. Search
c. Display

Section II: Database Management Systems-II **[15 Marks]**

Consider the following Project-Employee database maintained by a company which stores the details of the projects assigned to the employees.
Following are the tables:
PROJECT (pno integer, p_name char (30), ptype char(20),duration integer)
EMPLOYEE (eno integer, e_name char (20), qualification char (15), joindate date)

The relationships are as follows:

PROJECT-EMPLOYEE: M-M Relationship, with descriptive attributes as start_date (date), no_of_hours_worked (integer).

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message.

OR

1. Write a stored function to accept project name as input and print the names of employees working on the project. Raise an exception for an invalid project name.

2. Write a procedure to insert the values in project table. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>


struct node

{

   char name[30];

   char phone[15];

   struct node *left,*right;
```

```
};

/* Insert contact */
struct node* insert(struct node *T, char name[], char phone[])
{
    struct node *nw,*temp,*parent;

    nw=(struct node*)malloc(sizeof(struct node));
    strcpy(nw->name,name);
    strcpy(nw->phone,phone);
    nw->left=nw->right=NULL;

    if(T==NULL)
        return nw;

    temp=T;

    while(temp!=NULL)
    {
        parent=temp;

        if(strcmp(name,temp->name)<0)
            temp=temp->left;
        else
            temp=temp->right;
    }

    if(strcmp(name,parent->name)<0)
        parent->left=nw;
    else
        parent->right=nw;
```

```
    return T;
}

/* Search contact */
void search(struct node *T,char name[])
{
    while(T!=NULL)
    {
        if(strcmp(name,T->name)==0)
        {
            printf("Name: %s\nPhone: %s\n",T->name,T->phone);
            return;
        }

        if(strcmp(name,T->name)<0)
            T=T->left;
        else
            T=T->right;
    }

    printf("Contact not found\n");
}

/* Display phonebook */
void inorder(struct node *T)
{
    if(T!=NULL)
    {
        inorder(T->left);
        printf("%s : %s\n",T->name,T->phone);
```

```
        inorder(T->right);
    }
}

int main()
{
    struct node *T=NULL;
    int ch;
    char name[30],phone[15];

    while(1)
    {
        printf("\n1.Insert Contact");
        printf("\n2.Search Contact");
        printf("\n3.Display Phonebook");
        printf("\n4.Exit");

        printf("\nEnter choice: ");
        scanf("%d",&ch);

        switch(ch)
        {
            case 1:
                printf("Enter name: ");
                scanf("%s",name);

                printf("Enter phone number: ");
                scanf("%s",phone);

                T=insert(T,name,phone);
                break;
```

```
        case 2:
            printf("Enter name to search: ");
            scanf("%s",name);
            search(T,name);
            break;

        case 3:
            printf("\nPhonebook:\n");
            inorder(T);
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
    }
  }
}
```

DBMS:-

```
CREATE TABLE PROJECT (
  pno INTEGER PRIMARY KEY,
  p_name CHAR(30),
  ptype CHAR(20),
  duration INTEGER
);

CREATE TABLE EMPLOYEE (
  eno INTEGER PRIMARY KEY,
```

```
    e_name CHAR(20),
    qualification CHAR(15),
    joindate DATE
);
CREATE TABLE PROJECT_EMPLOYEE (
    pno INTEGER REFERENCES PROJECT(pno),
    eno INTEGER REFERENCES EMPLOYEE(eno),
    start_date DATE,
    no_of_hours_worked INTEGER,
    PRIMARY KEY(pno, eno)
);
```

1A)

```
CREATE OR REPLACE FUNCTION check_duration()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.duration <= 0 THEN
        RAISE EXCEPTION 'Duration must be greater than zero';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_duration
BEFORE INSERT
ON PROJECT
FOR EACH ROW
EXECUTE FUNCTION check_duration();
```

2)

```
LANGUAGE plpgsql
AS $$
BEGIN

    INSERT INTO Project(pno, pname, ptype, duration)
    VALUES (p_no, p_name, p_type, dur);

    RAISE NOTICE 'Project inserted successfully';

END;
$$;
```

Slip 15:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to implement a hash table using **Open Addressing with Quadratic Probing**. Perform the following operations. (Assume all keys are positive integers)

a. Insert
b. Search
c. Display

OR

Q1) Implement a Binary search tree (BST) library (btree.h) with operations – create, search, insert, inorder, preorder and postorder. Write a menu driven program that performs the above operations.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Person–Area database

Person (pnumber integer, pname varchar (20), birthdate date, income money)
Area (aname varchar (20), area-type varchar (5))
The relationships are as follows:

Person-Area: M-1
The attribute 'area_type' can have values either 'urban' or 'rural'.

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a cursor to update the income of all people living in 'Urban' area by 10%.

OR

1. Write a trigger before insert the record of person if the pnumber is negative give the message "Invalid Number".
2. Write a procedure to display all odd numbers from 1 to 100. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

"Btree.h"

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};
```

```
/* create first node */
struct node *create(struct node *root)
{
    struct node *nw,*temp;
    int n,i,num;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    temp=root;
    printf("Enter the nodes: ");

    for(i=0;i<n;i++)
    {
        nw=(struct node *)malloc(sizeof(struct node));
        nw->left=NULL;
        nw->right=NULL;
        scanf("%d",&num);
        nw->data=num;

        if(temp==NULL)
        {
            root=temp=nw;
        }
        else{
            temp=root;
            while(1)
            {
                if(nw->data <temp->data)
                {
                    if(temp->left == NULL)
```

```
                {
                    temp->left=nw;
                    break;
                }
                else{
                    temp=temp->left;
                }
            }
            else{
                if(temp->right == NULL)
                {
                    temp->right=nw;
                    break;
                }
                else{
                    temp=temp->right;
                }
            }
        }
    }
  }
  return root;
}

/* insert node */
struct node *insert(struct node *T, int x)
{
struct node *nw,*temp;
nw=(struct node*)malloc(sizeof(struct node));
nw->left=NULL;
nw->right=NULL;
```

```
nw->data=x;

if(T==NULL)
   T=nw;
else
   {
      temp=T;
      while(1)
      {
         if(x<temp->data)
            {
               if(temp->left==NULL)
               {
                  temp->left=nw;
                  break;
               }
               else{
                  temp=temp->left;
               }
            }
         else{
            if(temp->right==NULL)
            {
               temp->right=nw;
               break;
            }
            else{
            temp=temp->right;
            }
         }
      }
```

```
    }
return T;
}

/* search node */
struct node* search(struct node *T,int key)
{
    while(T!=NULL)
    {
        if(T->data==key)
            return T;

        if(key < T->data)
            T=T->left;
        else
            T=T->right;
    }

    return NULL;
}

/* inorder traversal */
void inorder(struct node *T)
{
    if(T!=NULL)
    {
        inorder(T->left);
        printf("%d ",T->data);
        inorder(T->right);
    }
}
```

```
/* preorder traversal */
void preorder(struct node *T)
{
   if(T!=NULL)
   {
      printf("%d ",T->data);
      preorder(T->left);
      preorder(T->right);
   }
}

/* postorder traversal */
void postorder(struct node *T)
{
   if(T!=NULL)
   {
      postorder(T->left);
      postorder(T->right);
      printf("%d ",T->data);
   }
}
```

Main.c

```
#include<stdio.h>
#include"btree.h"

int main()
{
   struct node *T=NULL,*temp;
```

```
int ch,x;

while(1)
{
   printf("\n1.Create");
   printf("\n2.Insert");
   printf("\n3.Search");
   printf("\n4.Inorder");
   printf("\n5.Preorder");
   printf("\n6.Postorder");
   printf("\n7.Exit");

   printf("\nEnter choice: ");
   scanf("%d",&ch);

   switch(ch)
   {
      case 1:
         root=create(root);
         break;

      case 2:
         printf("Enter value: ");
         scanf("%d",&x);
         T=insert(T,x);
         break;

      case 3:
         printf("Enter value to search: ");
         scanf("%d",&x);
```

```
            temp=search(T,x);

            if(temp==NULL)
                printf("Element not found\n");
            else
                printf("Element found\n");
            break;

        case 4:
            printf("Inorder Traversal:\n");
            inorder(T);
            break;

        case 5:
            printf("Preorder Traversal:\n");
            preorder(T);
            break;

        case 6:
            printf("Postorder Traversal:\n");
            postorder(T);
            break;

        case 7:
            return 0;

        default:
            printf("Invalid choice\n");
    }
  }
}
```

DBMS:-

```
CREATE TABLE Area (
   aname VARCHAR(20) PRIMARY KEY,
   area_type VARCHAR(5) CHECK (area_type IN ('urban','rural'))
);

CREATE TABLE Person (
   pnumber INTEGER PRIMARY KEY,
   pname VARCHAR(20),
   birthdate DATE,
   income MONEY,
   aname VARCHAR(20) REFERENCES Area(aname)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_pnumber()
RETURNS TRIGGER
AS $$
BEGIN

   IF NEW.pnumber < 0 THEN
      RAISE EXCEPTION 'Invalid Number';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_person_check
BEFORE INSERT
ON Person
FOR EACH ROW
EXECUTE FUNCTION check_pnumber();
```

2)

```
CREATE OR REPLACE PROCEDURE odd_numbers()
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 1;
BEGIN

   WHILE i <= 100 LOOP
      IF i % 2 <> 0 THEN
         RAISE NOTICE '%', i;
      END IF;

      i := i + 1;
   END LOOP;

END;
$$;
```

Slip 16:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a program to sort n randomly generated elements using Heapsort method.

OR

Q1) Write a C program for the implementation of Dijkstra's shortest path algorithm for finding shortest path from a given source vertex using adjacency cost matrix.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Student –Subject database

Student(rollno integer, name varchar(30), address varchar(50), class varchar(10))

Subject(scode varchar(10), subject_name varchar(20))

Student-Subject are related with **M-M** relationship with attributes **marks_scored.**

Q2) Using above Database Solve the following **[10 Marks]**

1) Define a trigger before deleting a student record from the student table. Raise a notice and display the message "student record is being deleted".

OR

1) Write a stored function using cursors, to calculate total marks of each student and display it.

2) Write a procedure to check the number is positive and negative. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#include<stdlib.h>

#include<time.h>


/* swap function */

void swap(int *a,int *b)

{

   int temp=*a;

   *a=*b;

   *b=temp;

}
```

```
/* heapify function */
void heapify(int arr[],int n,int i)
{
   int largest=i;
   int left=2*i+1;
   int right=2*i+2;

   if(left<n && arr[left]>arr[largest])
      largest=left;

   if(right<n && arr[right]>arr[largest])
      largest=right;

   if(largest!=i)
   {
      swap(&arr[i],&arr[largest]);
      heapify(arr,n,largest);
   }
}

/* heap sort */
void heapsort(int arr[],int n)
{
   int i;

   for(i=n/2-1;i>=0;i--)
      heapify(arr,n,i);

   for(i=n-1;i>=0;i--)
   {
      swap(&arr[0],&arr[i]);
```

```
        heapify(arr,i,0);
    }
}

int main()
{
    int n,i,arr[50];

    printf("Enter number of elements: ");
    scanf("%d",&n);

    srand(time(NULL));

    printf("Random numbers:\n");

    for(i=0;i<n;i++)
    {
        arr[i]=rand()%100;
        printf("%d ",arr[i]);
    }

    heapsort(arr,n);

    printf("\nSorted elements:\n");

    for(i=0;i<n;i++)
        printf("%d ",arr[i]);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Student (
   rollno INTEGER PRIMARY KEY,
   name VARCHAR(30),
   address VARCHAR(50),
   class VARCHAR(10)
);

CREATE TABLE Subject (
   scode VARCHAR(10) PRIMARY KEY,
   subject_name VARCHAR(20)
);

CREATE TABLE Student_Subject (
   rollno INTEGER REFERENCES Student(rollno),
   scode VARCHAR(10) REFERENCES Subject(scode),
   marks_scored INTEGER,
   PRIMARY KEY(rollno, scode)
);
```

1A)

```
CREATE OR REPLACE FUNCTION student_delete_msg()
RETURNS TRIGGER
AS $$
BEGIN
   RAISE NOTICE 'Student record is being deleted';
   RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_student_delete
```

```
BEFORE DELETE
ON Student
FOR EACH ROW
EXECUTE FUNCTION student_delete_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE check_number(n INTEGER)
LANGUAGE plpgsql
AS $$
BEGIN

   IF n > 0 THEN
      RAISE NOTICE 'Number is Positive';
   ELSIF n < 0 THEN
      RAISE NOTICE 'Number is Negative';
   ELSE
      RAISE NOTICE 'Number is Zero';
   END IF;

END;
$$;
```

Slip 17:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program that accepts the vertices and edges of a graph. Create and display adjacency list also print indegree, outdegree and total degree of all vertex of graph.

OR

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree.

Section II: Database Management Systems II **[15 Marks]**

Consider the following database maintained by a school about students and competitions.
STUDENT (sreg_no int , name char(30), class char(10))
COMPETITION (c_no int , name char(20), C_type char(15))

The relationship is as follows:

STUDENT-COMPETITION: **M-M** with described attributes rank , year and prize(int).

Q2) Using above Database Solve the following **[10 Marks]**

1) Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

OR

1) Write a function using cursor which accepts a name of student and returns the total no of prizes won by that student in the year 2020.
2) Write a procedure to find sum of first 50 numbers (using unconditional loop).

[5 marks]

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};


/* create node in BST */

struct node* create(struct node *T,int x)
```

```
{
   struct node *nw;

   if(T==NULL)
   {
      nw=(struct node*)malloc(sizeof(struct node));
      nw->data=x;
      nw->left=nw->right=NULL;
      return nw;
   }

   if(x < T->data)
      T->left=create(T->left,x);
   else
      T->right=create(T->right,x);

   return T;
}

/* find minimum */
int findMin(struct node *T)
{
   while(T->left!=NULL)
      T=T->left;

   return T->data;
}

/* find maximum */
int findMax(struct node *T)
{
```

```
    while(T->right!=NULL)
        T=T->right;

    return T->data;
}

int main()
{
    struct node *T=NULL;
    int n,i,x;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Enter value: ");
        scanf("%d",&x);
        T=create(T,x);
    }

    printf("Minimum value = %d\n",findMin(T));
    printf("Maximum value = %d\n",findMax(T));

    return 0;
}
```

DBMS:-

```
CREATE TABLE STUDENT (
    sreg_no INT PRIMARY KEY,
    name CHAR(30),
```

```
    class CHAR(10)
);

CREATE TABLE COMPETITION (
    c_no INT PRIMARY KEY,
    name CHAR(20),
    c_type CHAR(15)
);

CREATE TABLE STUDENT_COMPETITION (
    sreg_no INT REFERENCES STUDENT(sreg_no),
    c_no INT REFERENCES COMPETITION(c_no),
    rank INT,
    year INT,
    prize INT,
    PRIMARY KEY(sreg_no, c_no, year)
);
```

1A)

```
CREATE OR REPLACE FUNCTION comp_update_msg()
RETURNS TRIGGER
AS $$
BEGIN
    RAISE NOTICE 'Competition record is being updated';
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_comp_update
BEFORE UPDATE
ON COMPETITION
```

```
FOR EACH ROW
EXECUTE FUNCTION comp_update_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE sum_50()
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 1;
   s INT := 0;
BEGIN

   LOOP
      EXIT WHEN i > 50;

      s := s + i;
      i := i + 1;

   END LOOP;

   RAISE NOTICE 'Sum of first 50 numbers = %', s;

END;
$$;
```

Slip 18:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Item-Supplier database

Item(itemno integer, itemname varchar(20))

Supplier(supplier_no integer, supplier_name varchar(20), city varchar(20))

The relationship is as,

Item-Supplier: M-M relationship with rate(money) and quantity (integer) as descriptive attributes.

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a cursor to display the names of items whose rate is more than 500.

OR

1. Write a trigger before insert or update on rate field. If the rate is less than 50 then raise the appropriate exception.

2. Write a procedure to find the maximum number from three numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>


void cremat(int m[10][10], int n)

{

   int i,j;


   for(i=0;i<n;i++)

   {

      for(j=0;j<n;j++)

      {

         m[i][j]=0;
```

```
            if(i!=j)
            {
                printf("\nIs the edge present between %d & %d (1/0): ",i+1,j+1);
                scanf("%d",&m[i][j]);
            }
        }
    }
}

void dismat(int m[10][10], int n)
{
    int i,j;

    printf("\nThe Adjacency matrix is:\n");

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            printf("%d ",m[i][j]);
        }
        printf("\n");
    }
}

int main()
{
    int m[10][10], n;

    printf("Enter the number of vertices: ");
    scanf("%d",&n);
```

```
    cremat(m,n);
    dismat(m,n);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Item (
    itemno INTEGER PRIMARY KEY,
    itemname VARCHAR(20)
);

CREATE TABLE Supplier (
    supplier_no INTEGER PRIMARY KEY,
    supplier_name VARCHAR(20),
    city VARCHAR(20)
);

CREATE TABLE Item_Supplier (
    itemno INTEGER REFERENCES Item(itemno),
    supplier_no INTEGER REFERENCES Supplier(supplier_no),
    rate MONEY,
    quantity INTEGER,
    PRIMARY KEY(itemno, supplier_no)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_rate()
RETURNS TRIGGER
```

```
AS $$
BEGIN

    IF NEW.rate < 50 THEN
        RAISE EXCEPTION 'Rate must be greater than or equal to 50';
    END IF;

    RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_rate
BEFORE INSERT OR UPDATE
ON Item_Supplier
FOR EACH ROW
EXECUTE FUNCTION check_rate();
```

2)

```
CREATE OR REPLACE PROCEDURE max_three(a INT, b INT, c INT)
LANGUAGE plpgsql
AS $$
DECLARE
    max_val INT;
BEGIN

    max_val := a;

    IF b > max_val THEN
        max_val := b;
```

```
    END IF;

    IF c > max_val THEN
        max_val := c;
    END IF;

    RAISE NOTICE 'Maximum number is %', max_val;

END;
$$;
```

Slip 19:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to maintain a phonebook using Binary Search Tree by name where each node contains contact name and phone number.

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an **adjacency list.** Implement function to traverse the graph using Breadth First Search (BFS) traversal.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships for Project-Employee database.

Project (pno integer, pname char (30), ptype char (20), duration integer)
Employee (eno integer, ename char (20), qualification char (15), joining_date date)

The Relationship between **Project and Employee** is **many to many** with descriptive attribute start_date date, no_of_hours_worked integer.
Constraints: Primary Key, duration should be greater than zero, pname should not be null.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a stored function to accept eno as input parameter and count number of projects on which that employee is working.

OR

1) Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message
2) Write a procedure to find minimum and maximum from two numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>


struct node

{

   char name[30];

   char phone[15];

   struct node *left,*right;

};


/* create node in BST */
```

```
struct node* create(struct node *T, char name[], char phone[])
{
   struct node *nw;

   if(T==NULL)
   {
      nw=(struct node*)malloc(sizeof(struct node));
      strcpy(nw->name,name);
      strcpy(nw->phone,phone);
      nw->left=nw->right=NULL;
      return nw;
   }

   if(strcmp(name,T->name) < 0)
      T->left=create(T->left,name,phone);
   else
      T->right=create(T->right,name,phone);

   return T;
}

/* search contact */
void search(struct node *T,char name[])
{
   if(T==NULL)
   {
      printf("Contact not found\n");
      return;
   }

   if(strcmp(name,T->name)==0)
```

```
    {
        printf("Name: %s\nPhone: %s\n",T->name,T->phone);
        return;
    }

    if(strcmp(name,T->name)<0)
        search(T->left,name);
    else
        search(T->right,name);
}

/* display phonebook */
void inorder(struct node *T)
{
    if(T!=NULL)
    {
        inorder(T->left);
        printf("%s : %s\n",T->name,T->phone);
        inorder(T->right);
    }
}

int main()
{
    struct node *T=NULL;
    int ch;
    char name[30],phone[15];

    while(1)
    {
        printf("\n1.Add Contact");
```

```
printf("\n2.Search Contact");
printf("\n3.Display Phonebook");
printf("\n4.Exit");

printf("\nEnter choice: ");
scanf("%d",&ch);

switch(ch)
{
   case 1:
      printf("Enter name: ");
      scanf("%s",name);

      printf("Enter phone: ");
      scanf("%s",phone);

      T=create(T,name,phone);
      break;

   case 2:
      printf("Enter name to search: ");
      scanf("%s",name);
      search(T,name);
      break;

   case 3:
      printf("\nPhonebook:\n");
      inorder(T);
      break;

   case 4:
```

```
            return 0;

        default:
            printf("Invalid choice\n");
    }
  }
}
```

DBMS:-

```
CREATE TABLE Project (
  pno INTEGER PRIMARY KEY,
  pname CHAR(30) NOT NULL,
  ptype CHAR(20),
  duration INTEGER CHECK (duration > 0)
);

CREATE TABLE Employee (
  eno INTEGER PRIMARY KEY,
  ename CHAR(20),
  qualification CHAR(15),
  joining_date DATE
);

CREATE TABLE Project_Employee (
  pno INTEGER REFERENCES Project(pno),
  eno INTEGER REFERENCES Employee(eno),
  start_date DATE,
  no_of_hours_worked INTEGER,
  PRIMARY KEY (pno, eno)
);
```

1A)

```
CREATE OR REPLACE FUNCTION count_projects(emp_no INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
   total INT;
BEGIN

   SELECT COUNT(*)
   INTO total
   FROM Project_Employee
   WHERE eno = emp_no;

   RETURN total;

END;
$$;
```

2)

```
CREATE OR REPLACE PROCEDURE min_max(a INT, b INT)
LANGUAGE plpgsql
AS $$
DECLARE
   min_val INT;
   max_val INT;
BEGIN

   IF a < b THEN
      min_val := a;
      max_val := b;
   ELSE
```

```
        min_val := b;
        max_val := a;
    END IF;

    RAISE NOTICE 'Minimum = %', min_val;
    RAISE NOTICE 'Maximum = %', max_val;

END;
$$;
```

Slip 20:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an **adjacency list.** Implement function to traverse the graph using Depth First Search (DFS) traversal.

OR

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Department-employee database.

Department (dno integer, dname varchar(20),city varchar(20))

Employee (eno integer, ename varchar(20), salary money)

Department and Employee are related with a **one to many** relationships.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a function to accept department name and display the maximum salary of an employee in that department.

OR

1) Write a trigger before insert/update on an employee record. Raise exception if salary <0.

2) Write a procedure to accept a number and display multiplication table of a number.

[5 marks]

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#include<stdlib.h>


#define MAX 10


struct node

{

   int vertex;

   struct node *next;

};


void recdfs(struct node *adj[MAX], int v)

{

   struct node *temp;
```

```
    static int visit[MAX]={0};

    visit[v]=1;
    printf("\tV[%d]",v+1);

    temp=adj[v];

    while(temp!=NULL)
    {
        if(visit[temp->vertex]==0)
            recdfs(adj,temp->vertex);

        temp=temp->next;
    }
}

int main()
{
    struct node *adj[MAX], *nw, *temp;
    int n,i,j,reply;

    printf("Enter the number of vertices: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        adj[i]=NULL;

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
```

```
            if(i!=j)
            {
                printf("\nIs the edge present between %d & %d (1/0): ",i+1,j+1);
                scanf("%d",&reply);

                if(reply==1)
                {
                    nw=(struct node*)malloc(sizeof(struct node));
                    nw->vertex=j;
                    nw->next=NULL;

                    if(adj[i]==NULL)
                        adj[i]=nw;
                    else
                    {
                        temp=adj[i];
                        while(temp->next!=NULL)
                            temp=temp->next;

                        temp->next=nw;
                    }
                }
            }
        }
    }

    printf("\nDFS Traversal:\n");
    recdfs(adj,0);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Department (
   dno INTEGER PRIMARY KEY,
   dname VARCHAR(20),
   city VARCHAR(20)
);

CREATE TABLE Employee (
   eno INTEGER PRIMARY KEY,
   ename VARCHAR(20),
   salary MONEY,
   dno INTEGER REFERENCES Department(dno)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_salary()
RETURNS TRIGGER
AS $$
BEGIN

   IF NEW.salary < 0 THEN
      RAISE EXCEPTION 'Salary cannot be negative';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_salary_check
```

```
BEFORE INSERT OR UPDATE
ON Employee
FOR EACH ROW
EXECUTE FUNCTION check_salary();
```

2)

```
CREATE OR REPLACE PROCEDURE multiplication_table(n INT)
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 1;
BEGIN

   WHILE i <= 10 LOOP
      RAISE NOTICE '% x % = %', n, i, n*i;
      i := i + 1;
   END LOOP;

END;
$$;
```

Slip 21:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the implementation of Topological sorting.

OR

Q1) Write a C program to implement a hash table using **open addressing with linear probing**. perform the following operations.(Assume all keys are positive integers)

a. Insert
b. Search
c. Display

Section II: Database Management Systems II **[15 Marks]**

Consider the following database

Doctor(d_no int, d_name varchar(30), specialization varchar(35), phone_no int, charges int)

Hospital(h_no int, h_name varchar(20), city varchar(10))

Doctor and Hospital are related with **many to one** relationship.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a function which will accept the name of the hospital and calculate the average charges of doctors visiting that hospital.

OR

1) Write a trigger before insert/update on Doctor. Raise exception if charges are <0.

2) Write a procedure to insert the values in Doctor table. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>

#define MAX 10


typedef struct {

   int data[MAX];

   int top;

}STACK;


void initStack(STACK *ps) {

   ps->top = -1;

}
```

```
void push(STACK *ps, int num)
{
    ps->top++;
    ps->data[ps->top] = num;
}

int pop(STACK *ps)
{
    return (ps->data[ps->top--]);
}

int isempty(STACK *ps)
{
    return (ps->top == -1);
}

int isfull(STACK *ps)
{
    return (ps->top == MAX - 1);
}

void topologicalSort(int m[10][10], int n)
{
    int i,j,v,w;
    int visited[10]={0};
    int indegree[10]={0};
    STACK s;
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
```

```
            indegree[i] += m[j][i];
        }
    }
    initStack(&s);

    while(1)
    {
        for(v=0;v<n;v++)
        {
            if((visited[v]==0) && (indegree[v]==0))
            {
                visited[v]=1;
                push(&s,v);
                printf("v[%d]\t",v+1);
            }
        }
        if(isempty(&s))
            break;
        v=pop(&s);
        for(w=0;w<n;w++)
        {
            if(m[v][w]==1)
                indegree[w]= indegree[w]-1;
        }
    }
}

int main()
{
    int m[10][10],n,i,j;
    printf("Enter the number of vertices:");
```

```
    scanf("%d",&n);
    printf("Enter the adjacency matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&m[i][j]);
        }
    }
    printf("Topological order of the vertices:\n");
    topologicalSort(m,n);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Hospital (
    h_no INT PRIMARY KEY,
    h_name VARCHAR(20),
    city VARCHAR(10)
);

CREATE TABLE Doctor (
    d_no INT PRIMARY KEY,
    d_name VARCHAR(30),
    specialization VARCHAR(35),
    phone_no INT,
    charges INT,
    h_no INT REFERENCES Hospital(h_no)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_charges()
RETURNS TRIGGER
AS $$
BEGIN

   IF NEW.charges < 0 THEN
      RAISE EXCEPTION 'Charges cannot be negative';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_charges_check
BEFORE INSERT OR UPDATE
ON Doctor
FOR EACH ROW
EXECUTE FUNCTION check_charges();
```

2)

```
CREATE OR REPLACE PROCEDURE insert_doctor(
   dno INT,
   dname VARCHAR,
   spec VARCHAR,
   phone INT,
   ch INT
)
```

```
LANGUAGE plpgsql
AS $$
BEGIN

    INSERT INTO Doctor(d_no, d_name, specialization, phone_no, charges)
    VALUES (dno, dname, spec, phone, ch);

    RAISE NOTICE 'Doctor record inserted successfully';

END;
$$;
```

Slip 22:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to implement a hash table using **Open Addressing with Quadratic Probing**. Perform the following operations. (Assume all keys are positive integers)

a. Insert
b. Search
c. Display

OR

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships for Project-Employee database.

Project (pno integer, pname char (30), ptype char (20), duration integer)
Employee (eno integer, ename char (20), qualification char (15), joining_date date)

The Relationship between **Project and Employee** is **many to many** with descriptive attribute start_date date, no_of_hours_worked integer.
Constraints: Primary Key, duration should be greater than zero, pname should not be null.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a stored function to accept eno as input parameter and count number of projects on which that employee is working.

OR

1) Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message

2) Write a procedure to display first 20 odd numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>


void cremat(int m[10][10], int n)
{
   int i,j;


   for(i=0;i<n;i++)
   {
      for(j=0;j<n;j++)
      {
         m[i][j]=0;
```

```
            if(i!=j)
            {
                printf("\nIs the edge present between %d & %d (1/0): ",i+1,j+1);
                scanf("%d",&m[i][j]);
            }
        }
    }
}

void dismat(int m[10][10], int n)
{
    int i,j;

    printf("\nAdjacency Matrix:\n");

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
            printf("%d ",m[i][j]);

        printf("\n");
    }
}

int main()
{
    int m[10][10], n;

    printf("Enter number of vertices: ");
    scanf("%d",&n);
```

```
    cremat(m,n);
    dismat(m,n);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Project (
    pno INTEGER PRIMARY KEY,
    pname CHAR(30) NOT NULL,
    ptype CHAR(20),
    duration INTEGER CHECK (duration > 0)
);

CREATE TABLE Employee (
    eno INTEGER PRIMARY KEY,
    ename CHAR(20),
    qualification CHAR(15),
    joining_date DATE
);

CREATE TABLE Project_Employee (
    pno INTEGER REFERENCES Project(pno),
    eno INTEGER REFERENCES Employee(eno),
    start_date DATE,
    no_of_hours_worked INTEGER,
    PRIMARY KEY(pno, eno)
);
```

1A)

```
CREATE OR REPLACE FUNCTION count_projects(emp_no INT)
```

```
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
   total INT;
BEGIN

   SELECT COUNT(*)
   INTO total
   FROM Project_Employee
   WHERE eno = emp_no;

   RETURN total;

END;
$$;
```

2)

```
CREATE OR REPLACE PROCEDURE odd_numbers()
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 1;
   count INT := 0;
BEGIN

   WHILE count < 20 LOOP
      RAISE NOTICE '%', i;
      i := i + 2;
      count := count + 1;
   END LOOP;
```

```
END;
$$;
```

Slip 23:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

OR

Q1) Write a C program for the implementation of Topological sorting.

Section II: Database Management Systems II **[15 Marks]**

Consider the following database

Car (c_no int, owner varchar(20), model varchr(10), color varchar(10)

Driver (driver_no int, driver_name varchar(20), license_no int, d_age int, salary float)

Car and Driver are related with **many to many** relationship

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a cursor which accepts the driver name and prints the details of all cars that this driver has driven, if the driver name is invalid, print an appropriate message.

OR

1) Write a trigger before insert/update on Driver. Raise exception if driver age is < 21.

2) Write a procedure to find sum of first 100 numbers (using unconditional loop).

[5 marks]

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#define MAX 10


typedef struct {

   int data[MAX];

   int top;

}STACK;


void initStack(STACK *ps) {

   ps->top = -1;

}


void push(STACK *ps, int num)

{

   ps->top++;
```

```
    ps->data[ps->top] = num;
}

int pop(STACK *ps)
{
    return (ps->data[ps->top--]);
}

int isempty(STACK *ps)
{
    return (ps->top == -1);
}

int isfull(STACK *ps)
{
    return (ps->top == MAX - 1);
}

void topologicalSort(int m[10][10], int n)
{
    int i,j,v,w;
    int visited[10]={0};
    int indegree[10]={0};
    STACK s;
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            indegree[i] += m[j][i];
        }
    }
```

```
    initStack(&s);

    while(1)
    {
        for(v=0;v<n;v++)
        {
            if((visited[v]==0) && (indegree[v]==0))
            {
                visited[v]=1;
                push(&s,v);
                printf("v[%d]\t",v+1);
            }
        }
        if(isempty(&s))
            break;
        v=pop(&s);
        for(w=0;w<n;w++)
        {
            if(m[v][w]==1)
                indegree[w]= indegree[w]-1;
        }
    }
}

int main()
{
    int m[10][10],n,i,j;
    printf("Enter the number of vertices:");
    scanf("%d",&n);
    printf("Enter the adjacency matrix:\n");
    for(i=0;i<n;i++)
```

```
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&m[i][j]);
        }
    }
    printf("Topological order of the vertices:\n");
    topologicalSort(m,n);

    return 0;
}
```

DBMS:-

```
CREATE TABLE Car (
    c_no INT PRIMARY KEY,
    owner VARCHAR(20),
    model VARCHAR(10),
    color VARCHAR(10)
);

CREATE TABLE Driver (
    driver_no INT PRIMARY KEY,
    driver_name VARCHAR(20),
    license_no INT,
    d_age INT,
    salary FLOAT
);

CREATE TABLE Car_Driver (
    c_no INT REFERENCES Car(c_no),
    driver_no INT REFERENCES Driver(driver_no),
```

```
    PRIMARY KEY (c_no, driver_no)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_driver_age()
RETURNS TRIGGER
AS $$
BEGIN

    IF NEW.d_age < 21 THEN
        RAISE EXCEPTION 'Driver age must be at least 21';
    END IF;

    RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_driver_age
BEFORE INSERT OR UPDATE
ON Driver
FOR EACH ROW
EXECUTE FUNCTION check_driver_age();
```

2)

```
CREATE OR REPLACE PROCEDURE sum_100()
LANGUAGE plpgsql
AS $$
```

```
DECLARE
   i INT := 1;
   s INT := 0;
BEGIN

   LOOP
      EXIT WHEN i > 100;

      s := s + i;
      i := i + 1;

   END LOOP;

   RAISE NOTICE 'Sum of first 100 numbers = %', s;

END;
$$;
```

Slip 24:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the implementation of Floyd Warshall's algorithm for finding all pairs shortest path using adjacency cost matrix.

OR

Q1) Write a C program to maintain a phonebook using Binary Search Tree by name where each node contains contact name and phone number.

Section II: Database Management Systems II **[15 Marks]**

Consider the following Department-employee database.

Department (dno integer, dname varchar(20),city varchar(20))

Employee (eno integer, ename varchar(20), salary money)

Department and Employee are related with a **one to many** relationship

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a function to accept department name as input and display employee name along with salary of that department.

OR

1) Write a trigger after insert on an employee record. Display appropriate message when the record is inserted.

2) Write a stored procedure to display first 20 even numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>


struct node

{

   char name[30];

   char phone[15];

   struct node *left,*right;

};


/* create node in BST */

struct node* create(struct node *T,char name[],char phone[])
```

```
{
    struct node *nw;

    if(T==NULL)
    {
        nw=(struct node*)malloc(sizeof(struct node));
        strcpy(nw->name,name);
        strcpy(nw->phone,phone);
        nw->left=nw->right=NULL;
        return nw;
    }

    if(strcmp(name,T->name)<0)
        T->left=create(T->left,name,phone);
    else
        T->right=create(T->right,name,phone);

    return T;
}

/* search contact */
void search(struct node *T,char name[])
{
    if(T==NULL)
    {
        printf("Contact not found\n");
        return;
    }

    if(strcmp(name,T->name)==0)
    {
```

```
        printf("Name: %s\nPhone: %s\n",T->name,T->phone);
        return;
    }

    if(strcmp(name,T->name)<0)
        search(T->left,name);
    else
        search(T->right,name);
}

/* display phonebook */
void inorder(struct node *T)
{
    if(T!=NULL)
    {
        inorder(T->left);
        printf("%s : %s\n",T->name,T->phone);
        inorder(T->right);
    }
}

int main()
{
    struct node *T=NULL;
    int ch;
    char name[30],phone[15];

    while(1)
    {
        printf("\n1.Add Contact");
        printf("\n2.Search Contact");
```

```
printf("\n3.Display Phonebook");
printf("\n4.Exit");

printf("\nEnter choice: ");
scanf("%d",&ch);

switch(ch)
{
   case 1:
      printf("Enter name: ");
      scanf("%s",name);

      printf("Enter phone number: ");
      scanf("%s",phone);

      T=create(T,name,phone);
      break;

   case 2:
      printf("Enter name to search: ");
      scanf("%s",name);
      search(T,name);
      break;

   case 3:
      printf("\nPhonebook:\n");
      inorder(T);
      break;

   case 4:
      return 0;
```

```
        default:
            printf("Invalid choice\n");
    }
  }
}
```

DBMS:-

```
CREATE TABLE Department (
  dno INTEGER PRIMARY KEY,
  dname VARCHAR(20),
  city VARCHAR(20)
);

CREATE TABLE Employee (
  eno INTEGER PRIMARY KEY,
  ename VARCHAR(20),
  salary MONEY,
  dno INTEGER REFERENCES Department(dno)
);
```

1B)

```
CREATE OR REPLACE FUNCTION emp_insert_msg()
RETURNS TRIGGER
AS $$
BEGIN
  RAISE NOTICE 'Employee record inserted successfully';
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_emp_insert
AFTER INSERT
ON Employee
FOR EACH ROW
EXECUTE FUNCTION emp_insert_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE even_numbers()
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 2;
   count INT := 0;
BEGIN

   WHILE count < 20 LOOP
      RAISE NOTICE '%', i;
      i := i + 2;
      count := count + 1;
   END LOOP;

END;
$$;
```

Slip 25:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the implementation of Dijkstra's shortest path algorithm for finding shortest path from a given source vertex using adjacency cost matrix.

OR

Q1) Implement a Binary search tree (BST) library (btree.h) with operations – create, search, insert, inorder, preorder and postorder. Write a menu driven program that performs the above operations.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following database

Customer (cno integer, cname varchar(20), city varchar(20))

Account (a_no int, a_type varchar(10), opening_date date, balance money)

Customer and Account are related with **one to many** relationship

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a function using cursor which accepts city name as input and prints the details of all customers in that city.

OR

1. Write a trigger which does not allow deletion of accounts of _____type.

2.Write a procedure to display all customers from 'Pune' city. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

"Btree.h"

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};


/* create first node */

struct node *create(struct node *root)

{
```

```
struct node *nw,*temp;
int n,i,num;

printf("Enter number of nodes: ");
scanf("%d",&n);

temp=root;
printf("Enter the nodes: ");

for(i=0;i<n;i++)
{
   nw=(struct node *)malloc(sizeof(struct node));
   nw->left=NULL;
   nw->right=NULL;
   scanf("%d",&num);
   nw->data=num;

   if(temp==NULL)
   {
      root=temp=nw;
   }
   else{
      temp=root;
      while(1)
      {
         if(nw->data <temp->data)
         {
            if(temp->left == NULL)
            {
               temp->left=nw;
               break;
```

```
                }
                else{
                    temp=temp->left;
                }
            }
            else{
                if(temp->right == NULL)
                {
                    temp->right=nw;
                    break;
                }
                else{
                    temp=temp->right;
                }
            }
        }
    }
  }
  return root;
}

/* insert node */
struct node *insert(struct node *T, int x)
{
struct node *nw,*temp;
nw=(struct node*)malloc(sizeof(struct node));
nw->left=NULL;
nw->right=NULL;
nw->data=x;

if(T==NULL)
```

```
    T=nw;
else
    {
        temp=T;
        while(1)
        {
            if(x<temp->data)
                {
                    if(temp->left==NULL)
                    {
                        temp->left=nw;
                        break;
                    }
                    else{
                        temp=temp->left;
                    }
                }
            else{
                if(temp->right==NULL)
                {
                    temp->right=nw;
                    break;
                }
                else{
                temp=temp->right;
                }
            }
        }
    }
return T;
}
```

```
/* search node */
struct node* search(struct node *T,int key)
{
   while(T!=NULL)
   {
      if(T->data==key)
         return T;

      if(key < T->data)
         T=T->left;
      else
         T=T->right;
   }

   return NULL;
}

/* inorder traversal */
void inorder(struct node *T)
{
   if(T!=NULL)
   {
      inorder(T->left);
      printf("%d ",T->data);
      inorder(T->right);
   }
}

/* preorder traversal */
void preorder(struct node *T)
```

```
{
   if(T!=NULL)
   {
      printf("%d ",T->data);
      preorder(T->left);
      preorder(T->right);
   }
}

/* postorder traversal */
void postorder(struct node *T)
{
   if(T!=NULL)
   {
      postorder(T->left);
      postorder(T->right);
      printf("%d ",T->data);
   }
}
```

Main.c

```
#include<stdio.h>
#include"btree.h"

int main()
{
   struct node *T=NULL,*temp;
   int ch,x;

   while(1)
```

```
{
    printf("\n1.Create");
    printf("\n2.Insert");
    printf("\n3.Search");
    printf("\n4.Inorder");
    printf("\n5.Preorder");
    printf("\n6.Postorder");
    printf("\n7.Exit");

    printf("\nEnter choice: ");
    scanf("%d",&ch);

    switch(ch)
    {
        case 1:
            root=create(root);
            break;

        case 2:
            printf("Enter value: ");
            scanf("%d",&x);
            T=insert(T,x);
            break;

        case 3:
            printf("Enter value to search: ");
            scanf("%d",&x);

            temp=search(T,x);

            if(temp==NULL)
```

```
            printf("Element not found\n");
        else
            printf("Element found\n");
        break;

    case 4:
        printf("Inorder Traversal:\n");
        inorder(T);
        break;

    case 5:
        printf("Preorder Traversal:\n");
        preorder(T);
        break;

    case 6:
        printf("Postorder Traversal:\n");
        postorder(T);
        break;

    case 7:
        return 0;

    default:
        printf("Invalid choice\n");
    }
  }
}
```

DBMS:-

```
CREATE TABLE Customer (
   cno INTEGER PRIMARY KEY,
   cname VARCHAR(20),
   city VARCHAR(20)
);

CREATE TABLE Account (
   a_no INT PRIMARY KEY,
   a_type VARCHAR(10),
   opening_date DATE,
   balance MONEY,
   cno INT REFERENCES Customer(cno)
);
```

1B)

```
CREATE OR REPLACE FUNCTION prevent_account_delete()
RETURNS TRIGGER
AS $$
BEGIN

   IF OLD.a_type = 'saving' THEN
      RAISE EXCEPTION 'Deletion of saving account is not allowed';
   END IF;

   RETURN OLD;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_delete
BEFORE DELETE
```

```
ON Account
FOR EACH ROW
EXECUTE FUNCTION prevent_account_delete();
```

2)

```
CREATE OR REPLACE PROCEDURE customers_pune()
LANGUAGE plpgsql
AS $$
DECLARE
   rec RECORD;
BEGIN

   FOR rec IN SELECT * FROM Customer WHERE city = 'Pune'
   LOOP
      RAISE NOTICE 'Customer No: %, Name: %, City: %',
                rec.cno, rec.cname, rec.city;
   END LOOP;

END;
$$;
```

Slip 26:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an **adjacency matrix**. Implement function to traverse the graph using Depth First Search (DFS) traversal.

OR

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree.

Section II: Database Management Systems II **[15 Marks]**

Consider the following database :

Item (itemno integer, Itemname varchar(20), quantity integer)

Supplier(supplierno integer ,Supplier name varchar(20),city varchar(20))

Item and supplier are related with many to many relationship. Rate is descriptive attribute.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a stored function using cursors, to accept Item name from the user and display the Rate and Supplier Name for that Item.

OR

2) Write a trigger before update on rate field. If the difference in the old rate and new rate is more than Rs 2000, raise an exception and display the corresponding message.

3) Write a procedure to display subtraction of three numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left,*right;

};


/* create BST */

struct node* create(struct node *T,int x)

{

   struct node *nw;
```

```
    if(T==NULL)
    {
        nw=(struct node*)malloc(sizeof(struct node));
        nw->data=x;
        nw->left=nw->right=NULL;
        return nw;
    }

    if(x < T->data)
        T->left=create(T->left,x);
    else
        T->right=create(T->right,x);

    return T;
}

/* find minimum */
int findMin(struct node *T)
{
    while(T->left!=NULL)
        T=T->left;

    return T->data;
}

/* find maximum */
int findMax(struct node *T)
{
    while(T->right!=NULL)
        T=T->right;
```

```
    return T->data;
}

int main()
{
    struct node *T=NULL;
    int n,i,x;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Enter value: ");
        scanf("%d",&x);
        T=create(T,x);
    }

    printf("Minimum value = %d\n",findMin(T));
    printf("Maximum value = %d\n",findMax(T));

    return 0;
}
```

DBMS:-

```
CREATE TABLE Item (
    itemno INTEGER PRIMARY KEY,
    itemname VARCHAR(20),
    quantity INTEGER
);
```

```
CREATE TABLE Supplier (
   supplierno INTEGER PRIMARY KEY,
   supplier_name VARCHAR(20),
   city VARCHAR(20)
);

CREATE TABLE Item_Supplier (
   itemno INTEGER REFERENCES Item(itemno),
   supplierno INTEGER REFERENCES Supplier(supplierno),
   rate MONEY,
   PRIMARY KEY(itemno, supplierno)
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_rate_difference()
RETURNS TRIGGER
AS $$
BEGIN

   IF ABS(NEW.rate - OLD.rate) > 2000 THEN
      RAISE EXCEPTION 'Rate difference cannot exceed Rs 2000';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_rate_update
BEFORE UPDATE
ON Item_Supplier
```

```
FOR EACH ROW
EXECUTE FUNCTION check_rate_difference();
```

2)

```
CREATE OR REPLACE PROCEDURE subtract_three(a INT, b INT, c INT)
LANGUAGE plpgsql
AS $$
DECLARE
   result INT;
BEGIN

   result := a - b - c;

   RAISE NOTICE 'Result = %', result;

END;
$$;
```

Slip 27:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the Implementation of Prim's Minimum spanning tree algorithm.

OR

Q1) Write a C program that accepts the vertices and edges of a graph and store it as an **adjacency matrix**. Implement function to traverse the graph using Breadth First Search (BFS) traversal.

Section II: Database Management Systems II **[15 Marks]**

Consider the following database :

Student (rollno integer, name varchar(30),class varchar(10))

Subject(Scode varchar(10),subject name varchar(20))

Student and subject are related with **M-M** relationship with attributes marks_scored.

Q2) Using above Database Solve the following **[10 Marks]**

1. Write a stored function using cursors, to accept class from the user and display the details of the students of that class.

OR

1. Write a trigger before insert/update the marks_scored. Raise exception if Marks are negative.
2. Write a procedure to display all even numbers from 1 to 50. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>


int a,b,u,v,n,i,j,e=1;

int visited[10]={0},min,mincost=0,cost[10][10];


void main()

{

   printf("Enter the number of vertices:");

   scanf("%d",&n);

   printf("Enter the cost adjacency matrix:\n");

   for(i=1;i<=n;i++)

   {

      for(j=1;j<=n;j++)

      {
```

```
        scanf("%d",&cost[i][j]);
        if(cost[i][j]==0)
            cost[i][j]=999;
    }
}
printf("\nCost matrix is:\n ");
for(i=1;i<=n;i++)
{
    for(j=1;j<=n;j++)
    {
        printf("%d\t",cost[i][j]);
    }
    printf("\n");
}

visited[1]=1;
while(e<n)
{
    for(i=1,min=999;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            if(cost[i][j]<min)
            {
                if(visited[i]!=0)
                {
                    min=cost[i][j];
                    a=u=i;
                    b=v=j;
                }
            }
```

```
            }
        }
        if(visited[u]==0 || visited[v]==0)
        {
            printf("\nEdge %d:(%d->%d) cost:%d",e++,a,b,min);
            mincost+=min;
            visited[b]=1;
        }
        cost[a][b]=cost[b][a]=999;
    }
    printf("\nMinimum cost=%d\n",mincost);
}
```

DBMS:-

```
CREATE TABLE Student (
    rollno INTEGER PRIMARY KEY,
    name VARCHAR(30),
    class VARCHAR(10)
);

CREATE TABLE Subject (
    scode VARCHAR(10) PRIMARY KEY,
    subject_name VARCHAR(20)
);

CREATE TABLE Student_Subject (
    rollno INTEGER REFERENCES Student(rollno),
    scode VARCHAR(10) REFERENCES Subject(scode),
    marks_scored INTEGER,
    PRIMARY KEY(rollno, scode)
```

```
);
```

1B)

```
CREATE OR REPLACE FUNCTION check_marks()
RETURNS TRIGGER
AS $$
BEGIN

   IF NEW.marks_scored < 0 THEN
      RAISE EXCEPTION 'Marks cannot be negative';
   END IF;

   RETURN NEW;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_marks_check
BEFORE INSERT OR UPDATE
ON Student_Subject
FOR EACH ROW
EXECUTE FUNCTION check_marks();
```

2)

```
CREATE OR REPLACE PROCEDURE even_numbers()
LANGUAGE plpgsql
AS $$
DECLARE
```

```
    i INT := 2;
BEGIN

    WHILE i <= 50 LOOP
        RAISE NOTICE '%', i;
        i := i + 2;
    END LOOP;

END;
$$;
```

Slip 28:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to implement a hash table using **Open Addressing with Quadratic Probing**. Perform the following operations. (Assume all keys are positive integers)

a. Insert
b. Delete
c. Display

OR

Q1) Write a C program for the Implementation of Prim's Minimum spanning tree algorithm.

Section II: Database Management Systems-II **[15 Marks]**

Consider the following database :

Company (Name varchar(30),address (50),city varchar(20), share_value money)

Person (pname varchar(30),pcity varchar (20))

Company and **Person** are related with **M to M** relationship with descriptive attribute No_of_shares

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a stored function to update the share_values by 20% for Person "_____".

OR

1) Write a trigger before deleting company record. Display appropriate message to the user.
2) Write a procedure to display division of two numbers use raise to display error messages.

[5 marks]

Q 3) Viva **[5 marks]**

Ds) 2)

```
#include<stdio.h>


int a,b,u,v,n,i,j,e=1;

int visited[10]={0},min,mincost=0,cost[10][10];


void main()

{

   printf("Enter the number of vertices:");

   scanf("%d",&n);

   printf("Enter the cost adjacency matrix:\n");

   for(i=1;i<=n;i++)

   {

      for(j=1;j<=n;j++)

      {
```

```
        scanf("%d",&cost[i][j]);
        if(cost[i][j]==0)
            cost[i][j]=999;
    }
}
printf("\nCost matrix is:\n ");
for(i=1;i<=n;i++)
{
    for(j=1;j<=n;j++)
    {
        printf("%d\t",cost[i][j]);
    }
    printf("\n");
}

visited[1]=1;
while(e<n)
{
    for(i=1,min=999;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            if(cost[i][j]<min)
            {
                if(visited[i]!=0)
                {
                    min=cost[i][j];
                    a=u=i;
                    b=v=j;
                }
            }
```

```
        }
    }
    if(visited[u]==0 || visited[v]==0)
    {
        printf("\nEdge %d:(%d->%d) cost:%d",e++,a,b,min);
        mincost+=min;
        visited[b]=1;
    }
    cost[a][b]=cost[b][a]=999;
  }
  printf("\nMinimum cost=%d\n",mincost);
}
```

DBMS:-

```
CREATE TABLE Company (
  name VARCHAR(30) PRIMARY KEY,
  address VARCHAR(50),
  city VARCHAR(20),
  share_value MONEY
);

CREATE TABLE Person (
  pname VARCHAR(30) PRIMARY KEY,
  pcity VARCHAR(20)
);

CREATE TABLE Company_Person (
  name VARCHAR(30) REFERENCES Company(name),
  pname VARCHAR(30) REFERENCES Person(pname),
  no_of_shares INTEGER,
  PRIMARY KEY(name, pname)
```

```
);
```

1B)

```
CREATE OR REPLACE FUNCTION company_delete_msg()
RETURNS TRIGGER
AS $$
BEGIN

    RAISE NOTICE 'Company record is being deleted';

    RETURN OLD;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_company_delete
BEFORE DELETE
ON Company
FOR EACH ROW
EXECUTE FUNCTION company_delete_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE divide_numbers(a INT, b INT)
LANGUAGE plpgsql
AS $$
DECLARE
    result NUMERIC;
BEGIN
```

```
    IF b = 0 THEN
        RAISE EXCEPTION 'Division by zero is not allowed';
    END IF;

    result := a / b;

    RAISE NOTICE 'Result = %', result;

END;
$$;
```

Slip 29:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a program to sort n randomly generated elements using Heapsort method.

OR

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

Section II: Database Management Systems II **[15 Marks]**

Consider the following database

Person (pno int, pname varchar (20), birthdate date, income money)

Area (aid int,aname varchar (20), area_type varchar (5))

The **person and area** related to **many to one** relationship. The attribute 'area_type' can have values either 'urban' or 'rural'.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a cursor to display the names of persons living in urban area.

OR

1) Write a trigger before deleting a person's record from the person's table. Raise a notice and display the message "person record is being deleted".
2) Write a procedure to display subtraction of three numbers. **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>


void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}


void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i >= 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int arr[20], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
```

```
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    heapSort(arr, n);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf(" %d ", arr[i]);
    return 0;
}
```

DBMS:-

```
CREATE TABLE Area (
    aid INT PRIMARY KEY,
    aname VARCHAR(20),
    area_type VARCHAR(5) CHECK (area_type IN ('urban','rural'))
);

CREATE TABLE Person (
    pno INT PRIMARY KEY,
    pname VARCHAR(20),
    birthdate DATE,
    income MONEY,
    aid INT REFERENCES Area(aid)
);
```

1B)

```
CREATE OR REPLACE FUNCTION person_delete_msg()
RETURNS TRIGGER
AS $$
BEGIN
```

```
    RAISE NOTICE 'Person record is being deleted';

    RETURN OLD;

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_person_delete
BEFORE DELETE
ON Person
FOR EACH ROW
EXECUTE FUNCTION person_delete_msg();
```

2)

```
CREATE OR REPLACE PROCEDURE subtract_three(a INT, b INT, c INT)
LANGUAGE plpgsql
AS $$
DECLARE
    result INT;
BEGIN

    result := a - b - c;

    RAISE NOTICE 'Result = %', result;

END;
$$;
```

Slip 30:-

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

OR

Q1) Write a C program to implement a hash table using **open addressing with linear probing**. perform the following operations. (Assume all keys are positive integers)

a.Insert
b.Search
c.Display

Section II: Database Management Systems II **[15 Marks]**

Consider the following database

Student (Roll_No int, Sname varchar (20), Sclass char (10))

Teacher (T_No int, Tname char (20), Experience int)

Student and Teacher are related with **many to many** relationship with the descriptive attribute Subject.

Q2) Using above Database Solve the following **[10 Marks]**

1) Write a stored function to count the number of teachers having experience > 10 years

OR

1) Write a trigger before insert the record of the student in the Student table. If the Roll_No is less than or equal to zero then the trigger gets fired and displays the message "Invalid Roll Number".
2) Write a procedure to find sum and average of first n numbers using conditional loop(while) **[5 marks]**

Q 3) Viva **[5 marks]**

Ds) 1)

```
#include<stdio.h>


int i,j,a,b,u,v,n,e=1;

int min,mincost=0,cost[10][10],parent[10];

int find(int);

int uni(int,int);


int main()

{

   printf("\nThe edge of maximum cost spanning tree are:\n");

   while(e<n)

   {

      for(i=1,min=999;i<=n;i++)
```

```
        {
            for(j=1;j<=n;j++)
            {
                if(cost[i][j]<min)
                {
                    min=cost[i][j];
                    a=u=i;
                    b=v=j;
                }
            }
        }
        u=find(u);
        v=find(v);
        if(uni(u,v))
        {
            printf("\nEdge %d:(%d->%d) cost:%d",e++,a,b,min);
            mincost+=min;
        }
        cost[a][b]=cost[b][a]=999;
    }
    printf("\nMinimum cost=%d\n",mincost);

    return 0;
}

int find(int i)
{
    while(parent[i])
        i=parent[i];
    return i;
}
```

```
int uni(int i,int j)
{
   if(i!=j)
   {
      parent[j]=i;
      return 1;
   }
   return 0;
}
```

DBMS:-

```
CREATE TABLE Student (
   Roll_No INT PRIMARY KEY,
   Sname VARCHAR(20),
   Sclass CHAR(10)
);

CREATE TABLE Teacher (
   T_No INT PRIMARY KEY,
   Tname CHAR(20),
   Experience INT
);

CREATE TABLE Student_Teacher (
   Roll_No INT REFERENCES Student(Roll_No),
   T_No INT REFERENCES Teacher(T_No),
   Subject VARCHAR(20),
   PRIMARY KEY(Roll_No, T_No)
);
```

1A)

```
CREATE OR REPLACE FUNCTION count_teachers()
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
   total INT;
BEGIN

   SELECT COUNT(*)
   INTO total
   FROM Teacher
   WHERE Experience > 10;

   RETURN total;

END;
$$;
```

2)

```
CREATE OR REPLACE PROCEDURE sum_avg_n(n INT)
LANGUAGE plpgsql
AS $$
DECLARE
   i INT := 1;
   s INT := 0;
   avg_val NUMERIC;
BEGIN

   WHILE i <= n LOOP
```

```
        s := s + i;
        i := i + 1;
    END LOOP;

    avg_val := s / n;

    RAISE NOTICE 'Sum = %', s;
    RAISE NOTICE 'Average = %', avg_val;

END;
$$;
```